

Lecture 5: Decoding



NYU

Based on slides from
Eunsol Choi

Logistics

- ▶ Quiz 2 grades have been released.
- ▶ HW2 due next Wednesday (10/7), HW3 will be released then.

Today

- ▶ Tips for Final Project Brainstorming
- ▶ Decoding with Language Model
- ▶ Variants of Transformer Language Model

Final Project

- ▶ Getting started with NLP research project
 - ▶ Where should we start?
 - ▶ **Dataset**
 - ▶ Evaluation
 - ▶ Model

Where to find dataset?

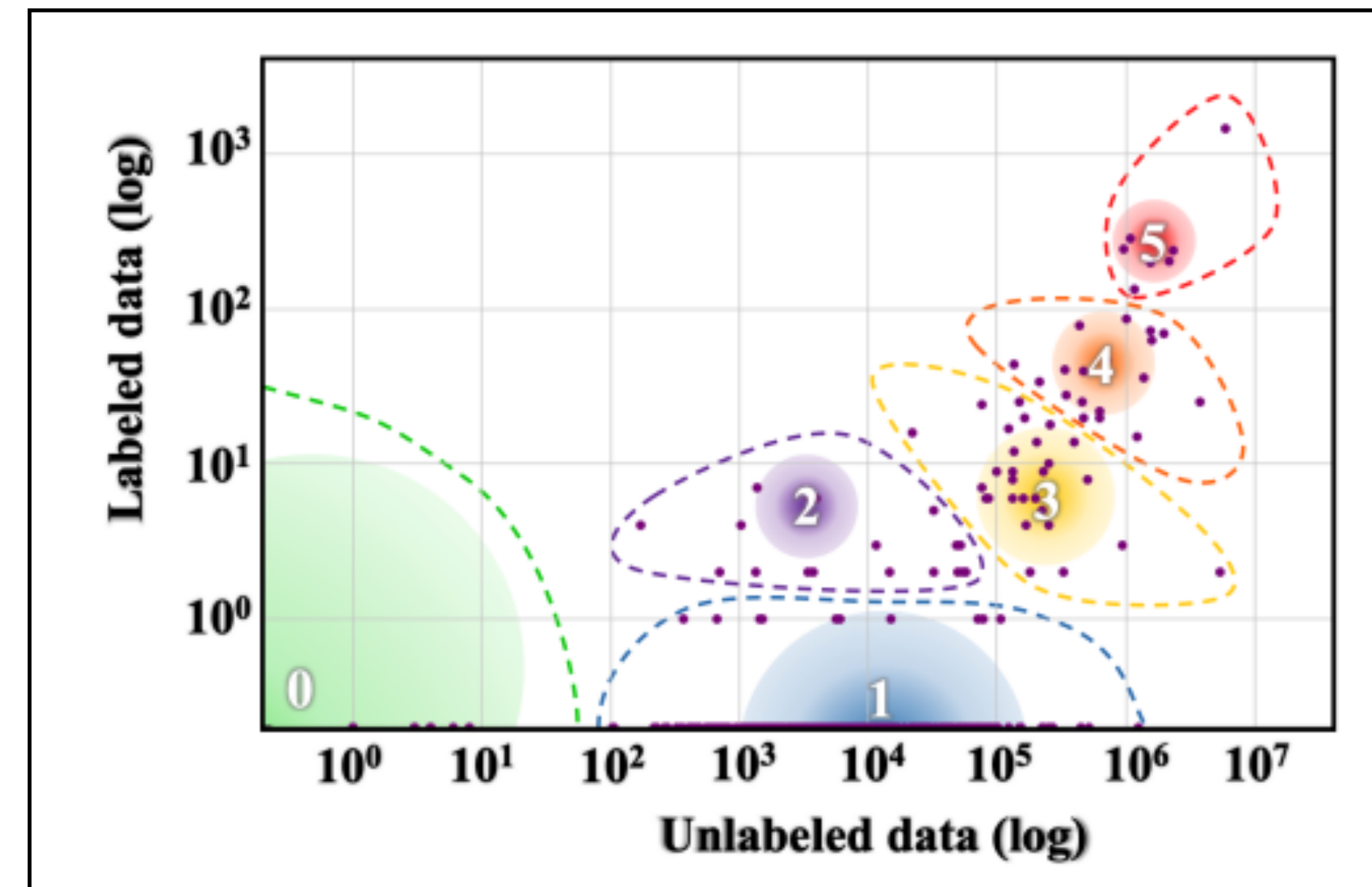
- ▶ Existing datasets
 - ▶ Linguistic Data Consortium
 - ▶ HuggingFace “Datasets”
- ▶ Find them in the wild
 - ▶ Example: StackOverflow, MovieReview datasets
 - ▶ Careful with copyright
- ▶ Build them
 - ▶ Collect dataset with experts
 - ▶ Crowdsourcing
 - ▶ But be careful with generating artificial data that does not reflect real world

Dataset: Exclusion

- ▶ Most of our annotated data is English data, especially newswire
- ▶ What about:
 - Other dialects of English?
 - Other languages? (Especially non-European/CJK)
 - Code switching? (Sentences which mixes multiple languages)
- ▶ If important technological tools don't work for some users, where does that leave those users?

Representation Disparity: Language

- Publications per language
- Labeled (and unlabeled) dataset distribution



Class	5 Example Languages	#Langs	#Speakers	% of Total Langs
0	Dahalo, Warlpiri, Popoloca, Wallisian, Bora	2191	1.2B	88.38%
1	Cherokee, Fijian, Greenlandic, Bhojpuri, Navajo	222	30M	5.49%
2	Zulu, Konkani, Lao, Maltese, Irish	19	5.7M	0.36%
3	Indonesian, Ukrainian, Cebuano, Afrikaans, Hebrew	28	1.8B	4.42%
4	Russian, Hungarian, Vietnamese, Dutch, Korean	18	2.2B	1.07%
5	English, Spanish, German, Japanese, French	7	2.5B	0.28%

Dataset

- ▶ Likely you will split your data into training set / development set / test set.
 - ▶ Typically random split
 - ▶ But if you are testing domain transfer, etc, you can do some other split.
- ▶ Many datasets will come with pre-defined splits — you should follow them if you'd like to compare it to existing work
- ▶ Don't plan on running multiple times on test data. (Supposed to be ran only once or twice!)

Be aware of biases in evaluation

Wason selection task

You are shown a set of four cards placed on a table, each of which has a number on one side and a color on the other.

Which card(s) must you turn over in order to test that **if a card shows an even number on one face, then its opposite face is blue**?



Be aware of biases in evaluation

The Wason 2-4-6 task

EXPERIMENTER SHOWS



PARTICIPANT TESTS

8, 10, 12

✓ fits

20, 22, 24

✓ fits

100, 102, 104

✓ fits

People are shown **2, 4, 6** and told it follows a secret rule.

They propose their own triples and hear whether each one fits.

When confident, they announce the rule.

Be aware of biases in evaluation

The Wason 2-4-6 task

EXPERIMENTER SHOWS



PARTICIPANT TESTS

8, 10, 12	✓ fits
20, 22, 24	✓ fits
100, 102, 104	✓ fits

People are shown **2, 4, 6** and told it follows a secret rule.

They propose their own triples and hear whether each one fits.

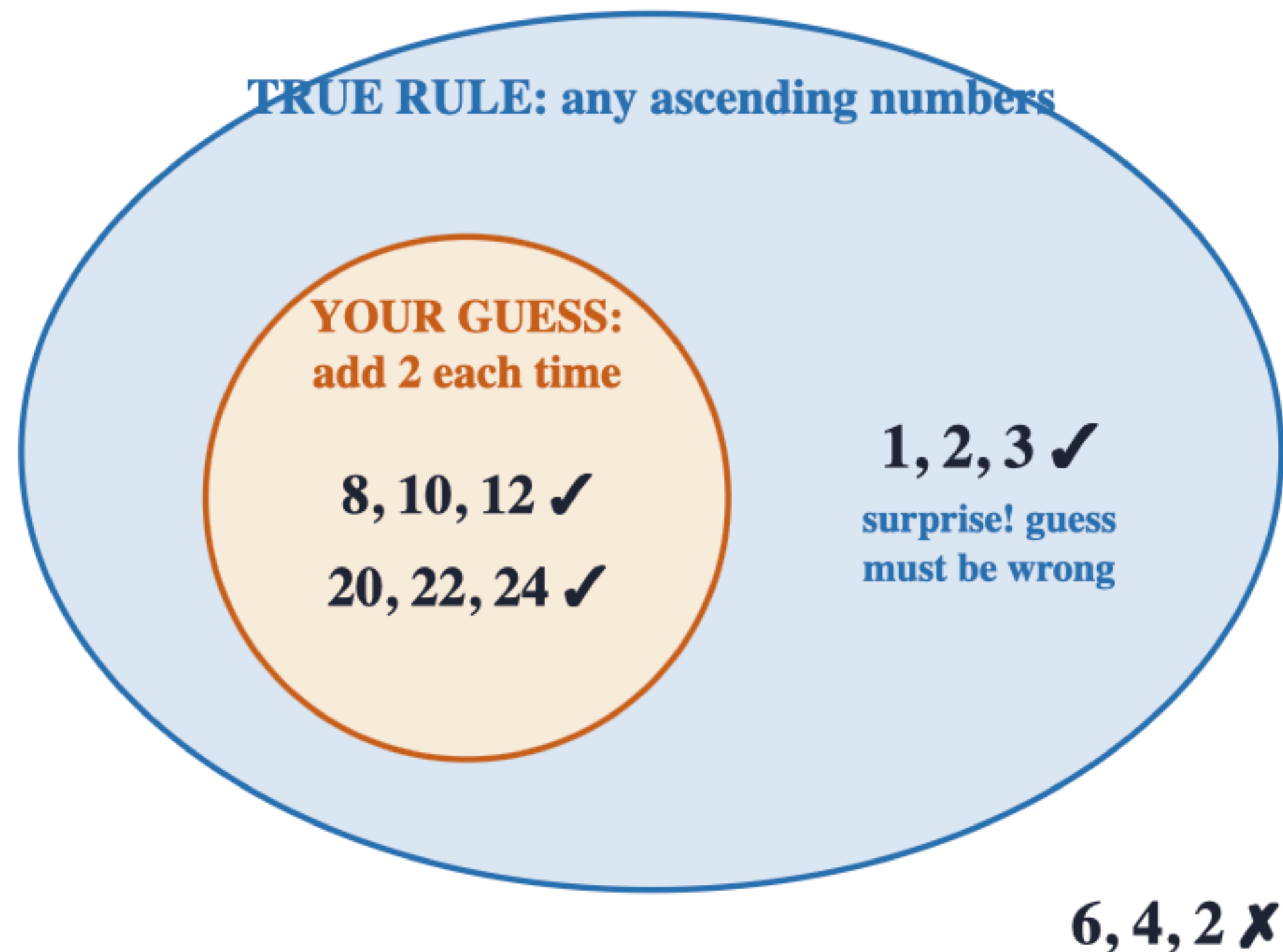
When confident, they announce the rule.

Typical answer: “**add 2 each time**”

Actual rule: **any three ascending numbers**

In evaluation, our goal is not just to find results that align with our hypothesis, but to actively try to **rule out alternative hypotheses**.

Be aware of biases in evaluation



Confirming tests

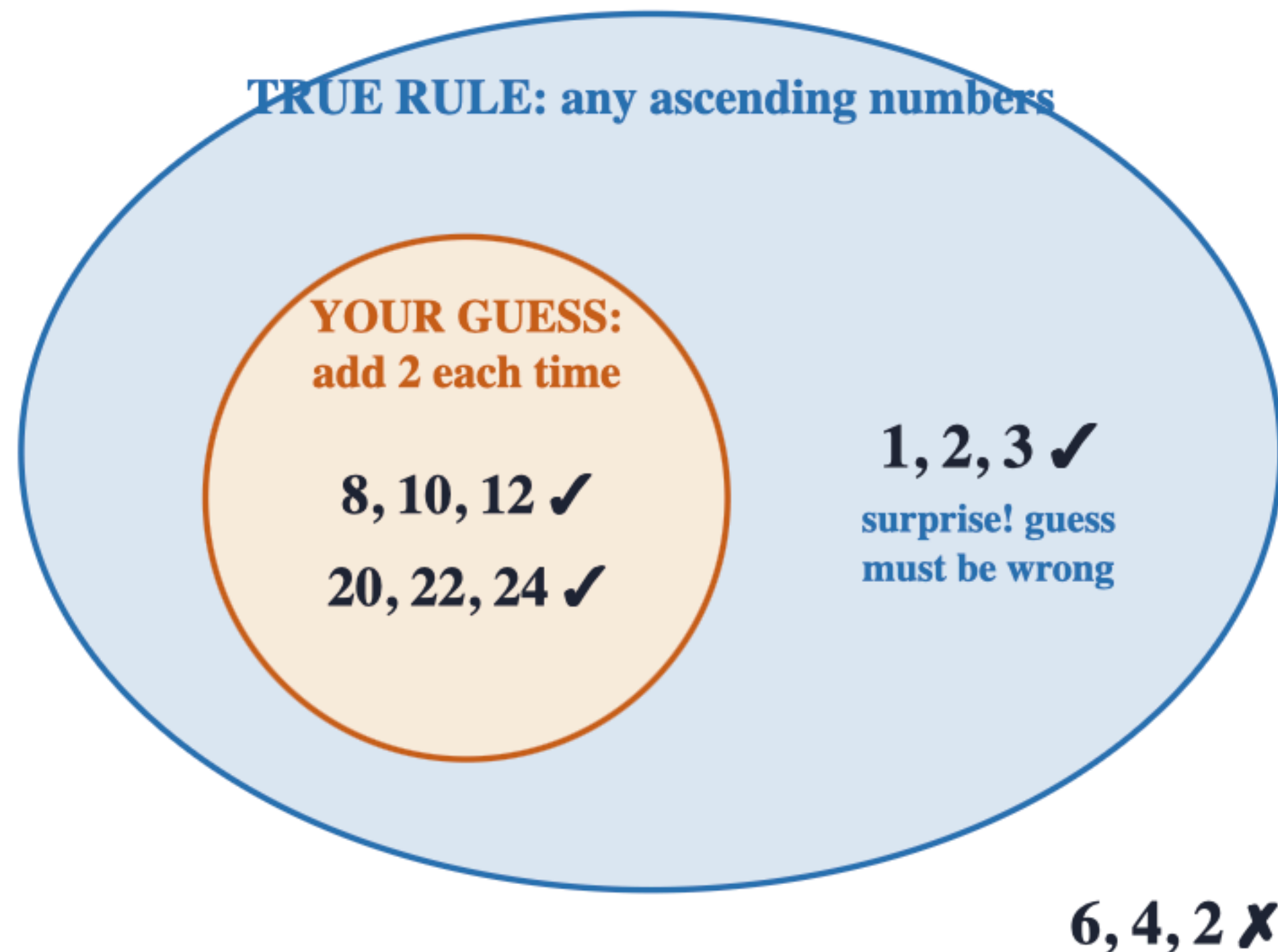
Triples that fit your guess always get “yes,” so you never learn it’s too narrow.

Disconfirming tests

Try triples your guess says *shouldn't* fit. A surprising answer reveals the real rule.

Lesson: we seek evidence that confirms our beliefs, not evidence that could refute them. This is **confirmation bias.**

Be aware of biases in evaluation



Confirming tests

Triples that fit your guess always get “yes,” so you never learn it’s too narrow.

Disconfirming tests

Try triples your guess says *shouldn't* fit. A surprising answer reveals the real rule.

Lesson: we seek evidence that confirms our beliefs, not evidence that could refute them. This is **confirmation bias.**

In evaluation, our goal is not just to find results that align with our hypothesis, but to actively try to **rule out alternative hypotheses**.

Quantitative Evaluation

- ▶ Follow prior work, use existing metrics
- ▶ Start with simple method, use ablations to study the effectiveness of your choices
- ▶ Consider smaller-scale human evaluation if possible, especially for generation task
- ▶ Test statistical significance when differences are small and data is small

Quantitative Evaluation

- ▶ Break down your performance number by various metrics!
 - ▶ Popular vs. rare entities?
 - ▶ Long vs. short sentences?
- ▶ Go beyond a single number comparison:
 - ▶ Is one method better than another in terms of precision, but not in recall?
 - ▶ Are they making mistakes on the same sets of examples?
- ▶ Reporting negative results can be useful too
 - ▶ But be extra careful with making strong statements — if it didn't work, is it because of potential bugs? Wrong hyperparameters?

Qualitative Analysis

- ▶ Goal: provide evidence that your hypothesis is (more) correct
- ▶ Often hard to evaluate with quantitative metrics:
 - ▶ e.g., Attention-based NMT models can learn the same kinds of alignments as phrase-based MT systems, but generalize better to unfamiliar words and phrases.
- ▶ Qualitative evidence can help
 - ▶ Be careful to not just cherry pick!
 - ▶ Take random samples, and categorize errors and count them
 - ▶ Visualize your model embeddings / etc with t-SNE
 - ▶ Build demo if you can!

NLP Leaderboards

SuperGLUE

GLUE

Paper

Code

Tasks

Leaderboard

FAQ

Diagnostics

Submit

Login

Rank

Name

Model

SQuAD2.0

The Stanford Question Answering Dataset

Leaderboard

SQuAD2.0 tests the ability of a system to not only answer reading comprehension questions, but also abstain when presented with a question that cannot be answered based on the provided paragraph.

Rank	Model	EM	F1
	Human Performance Stanford University (Rajpurkar & Jia et al. '18)	86.831	89.452
1	FPNet (ensemble) Ant Service Intelligence Team	90.871	93.183
2	IE-Net (ensemble) RICOH_SRCB_DML	90.758	93.044

Utility is in the Eye of the User: A Critique of NLP Leaderboards

Kawin Ethayarajh

Stanford University

kawin@stanford.edu

Dan Jurafsky

Stanford University

jurafsky@stanford.edu

4/96.4

91.8

84.6/54.7

89.0/88.3

88.8

74.1

93.2

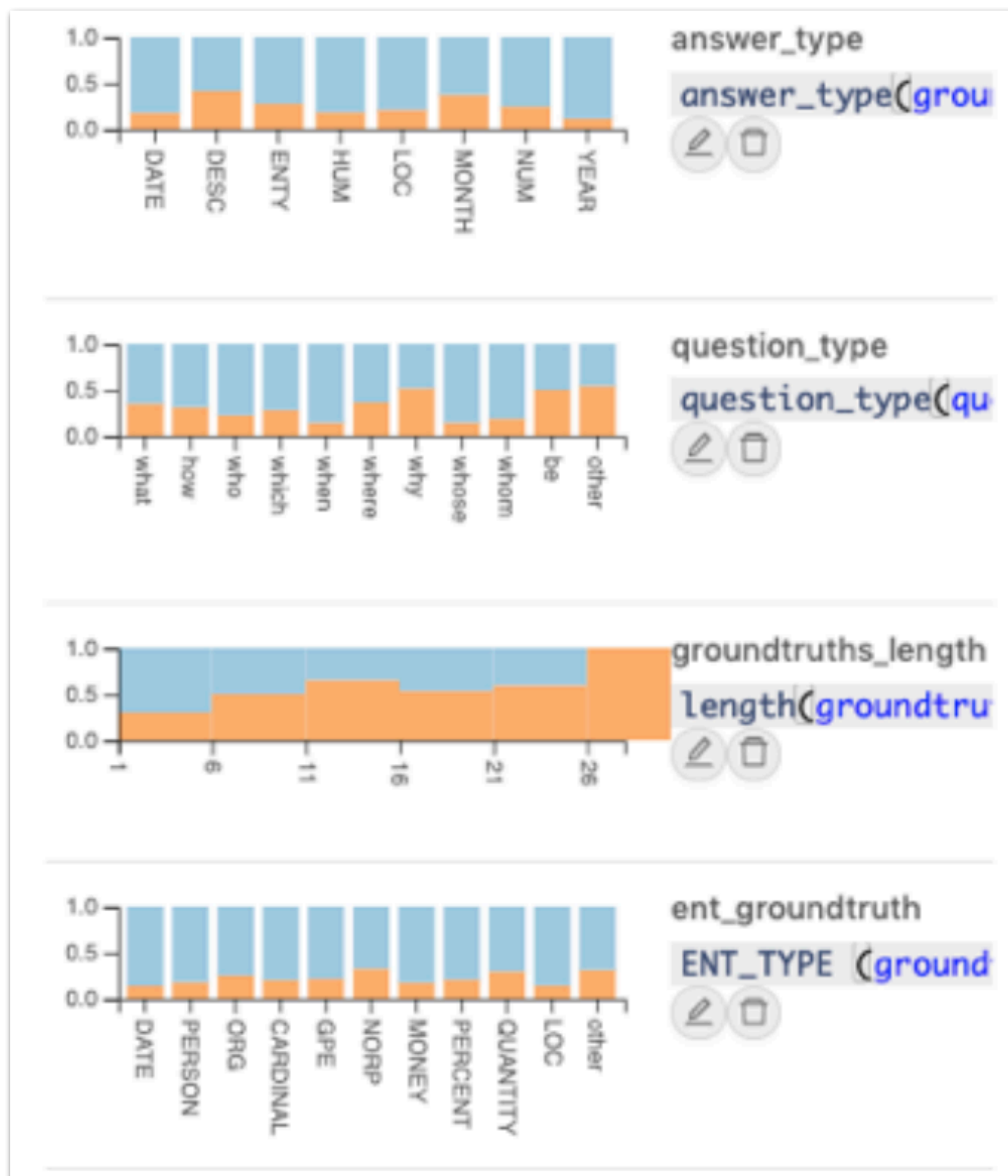
75.6

98.3/99.2

▶ Often focus on a single criteria — accuracy!

▶ Equitability across different demographics?

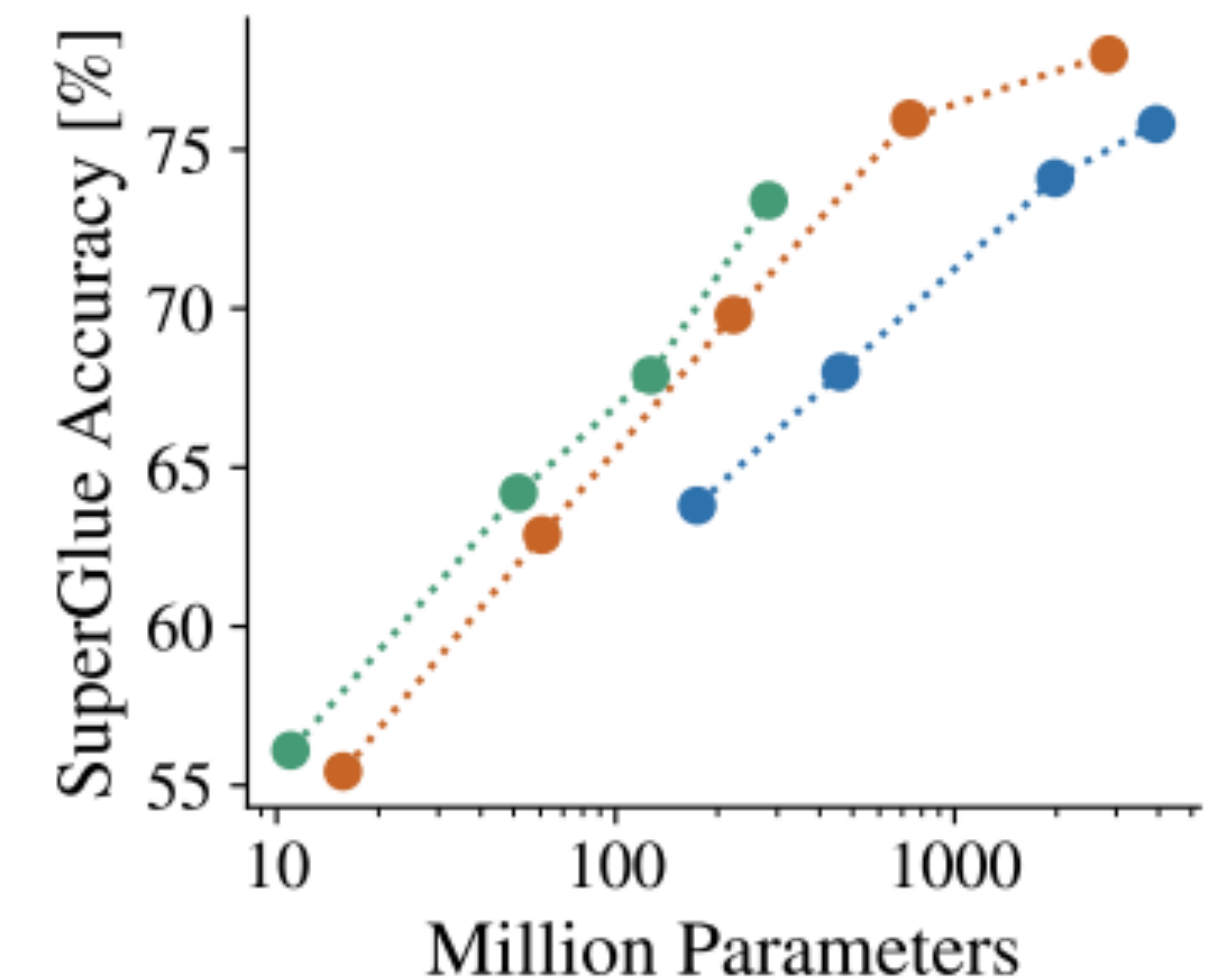
More nuanced leaderboards



- ▶ Understanding the error patterns
- ▶ Aligning with human values
 - ▶ Not all errors are equally damaging
- ▶ Bringing evaluation into the loop of model development — find examples where existing models fail, and only evaluate on them
 - ▶ Problems?

Scale vs. Efficiency

- ▶ In current era, larger models (with more parameters) tend to outperform smaller model
- ▶ How should we measure efficiency?
 - ▶ Training vs. Inference
 - ▶ Latency (# examples / seconds)
 - ▶ Memory requirements
 - ▶ # Flops



Today

- ▶ Getting started with NLP research project
 - ▶ Where should we start?
 - ▶ Dataset
 - ▶ Evaluation
 - ▶ **Model**

Model

- ▶ Build a simple baseline
 - ▶ e.g., Majority class label
- ▶ Build a strong baseline
 - ▶ Existing published work or large scale models (GPT5, etc) can be a good baseline
 - ▶ You don't necessarily have to beat them, especially if they are using a lot of resources that you do not have access to
- ▶ Motivate your model
 - ▶ In what aspect your proposed model improve upon baseline?

Hyperparameter Tuning

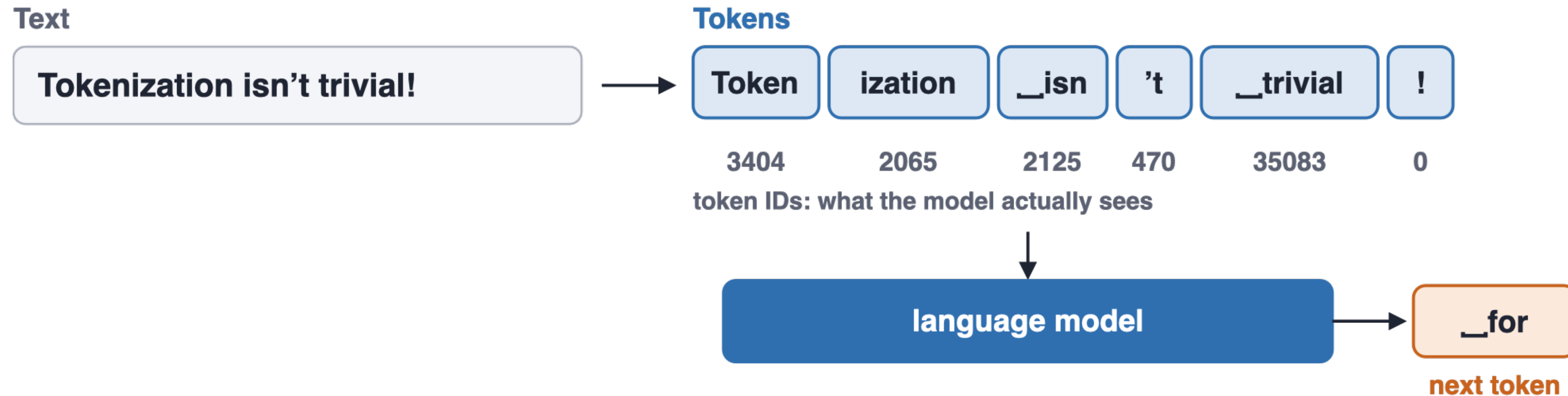
- ▶ You should tune both your baseline **AND** your new model
- ▶ During literature review, pay attention to what hyper parameters matter, and what are typical values

Today

- ▶ Tips for Final Project Brainstorming
- ▶ Tokenization
- ▶ Decoding with Language Model
 - ▶ Decoding Basics
 - ▶ Advanced Decoding methods
- ▶ Variants of Transformer Language Model

Tokenization

Language models read and write tokens



What is a token?

A frequent chunk of text: a whole word, part of a word, a space plus a word, or punctuation. `_` marks a leading space.

Why it matters today

Every decoding step in this lecture picks one **token**, not one word or character.

Tokenization

Why subwords? The middle ground

Characters

u n b e l i e v a b l y

Words

unbelievably

Subwords

un believ ably

Characters

12 tokens: tiny vocabulary, but long sequences and many decoding steps.

Words

1 token, only if the word is in the vocabulary. Rare words, typos and new names become [UNK].

Subwords

3 tokens: common words stay whole; rare words are built from common pieces.

Subword vocabularies are typically tens of thousands to a couple hundred thousand tokens.

Tokenization

Toy training text: only 4 different words, 16 in total

l o w × 5

appears 5 times

l o w e r × 2

appears 2 times

n e w e s t × 6

appears 6 times

w i d e s t × 3

appears 3 times

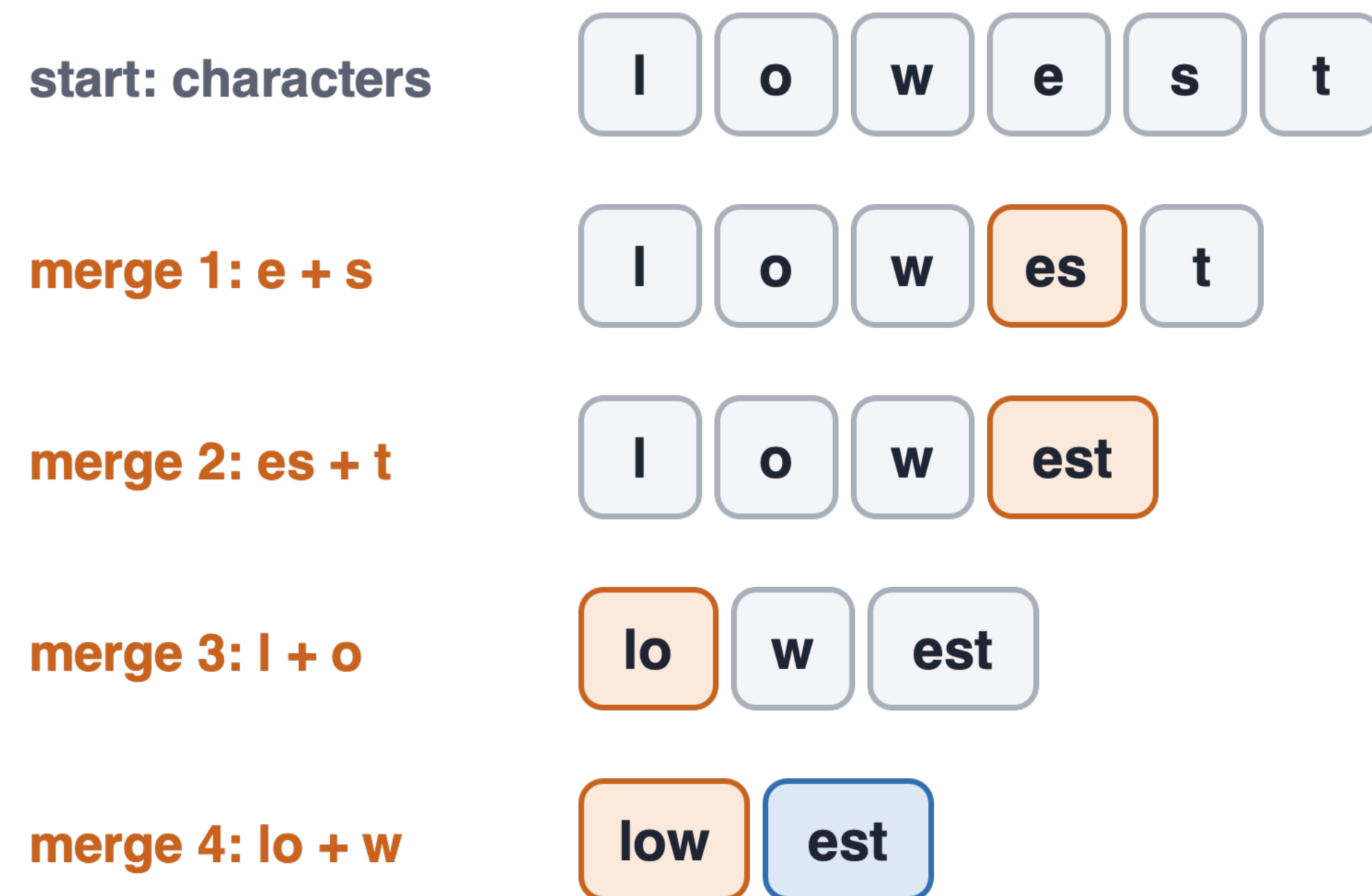
Rule: start with every word split into characters. Count every adjacent pair across all 16 words; merge the most frequent pair into a new token. Repeat.

Step	Merge	Times seen	Corpus afterwards: just merged · merged earlier
Start			l o w l o w e r n e w e s t w i d e s t
1	e + s	9 = 6+3	l o w l o w e r n e w es t w i d es t
2	es + t	9 = 6+3	l o w l o w e r n e w est w i d est
3	l + o	7 = 5+2	lo w lo w e r n e w est w i d est
4	lo + w	7 = 5+2	low low e r n e w est w i d est

Times seen = how often the pair occurs side by side in the 16 words: e + s occurs once in newest (x6) and once in widest (x3) = 9. Ties are broken arbitrarily. Sennrich et al. (2016).

Tokenization

Tokenizing a new word



Replay the merges, in the learned order
“lowest” never appeared in the corpus, yet it ends as two known tokens: **low** + **est**.

Nothing is unknown

In the worst case a word stays as single characters. Byte-level BPE starts from the 256 byte values, so any text can be encoded.

Tokenization

Tokenization has consequences

Cost

The model writes one token per step, so text that splits into more tokens takes more steps: slower and more expensive.

Languages

The same sentence can need several times more tokens in some languages: slower, costlier, less fits in the context.

Spelling

The model sees chunks, not letters, so “how many r’s in strawberry?” is harder than it looks.

straw

berry

Numbers

Digit grouping affects arithmetic. Many recent tokenizers split numbers into single digits or fixed groups.

123

45

vs

1

2

3

4

5

Tokenization

Why more tokens cost more

Word-like tokens
k = 6



Character tokens
k = 23



each tile = one token = one sequential forward pass of the model

More passes, in a row

Each generated token is one full pass through all L layers, and passes can't overlap: **6 vs 23** (3.8× the waiting).

Each pass attends to more

Every token attends to all earlier tokens, so attention work grows like k^2 : **36 vs 529** (14.7×).

Everything else is counted in tokens

KV cache size, context-window limits, and API prices all scale with the token count.

Tokenizers in real models

Model	Tokenizer	Vocabulary
GPT-2 (2019)	byte-level BPE	50,257
Llama 2 (2023)	SentencePiece BPE	32,000
Llama 3 (2024)	byte-level BPE	128,256

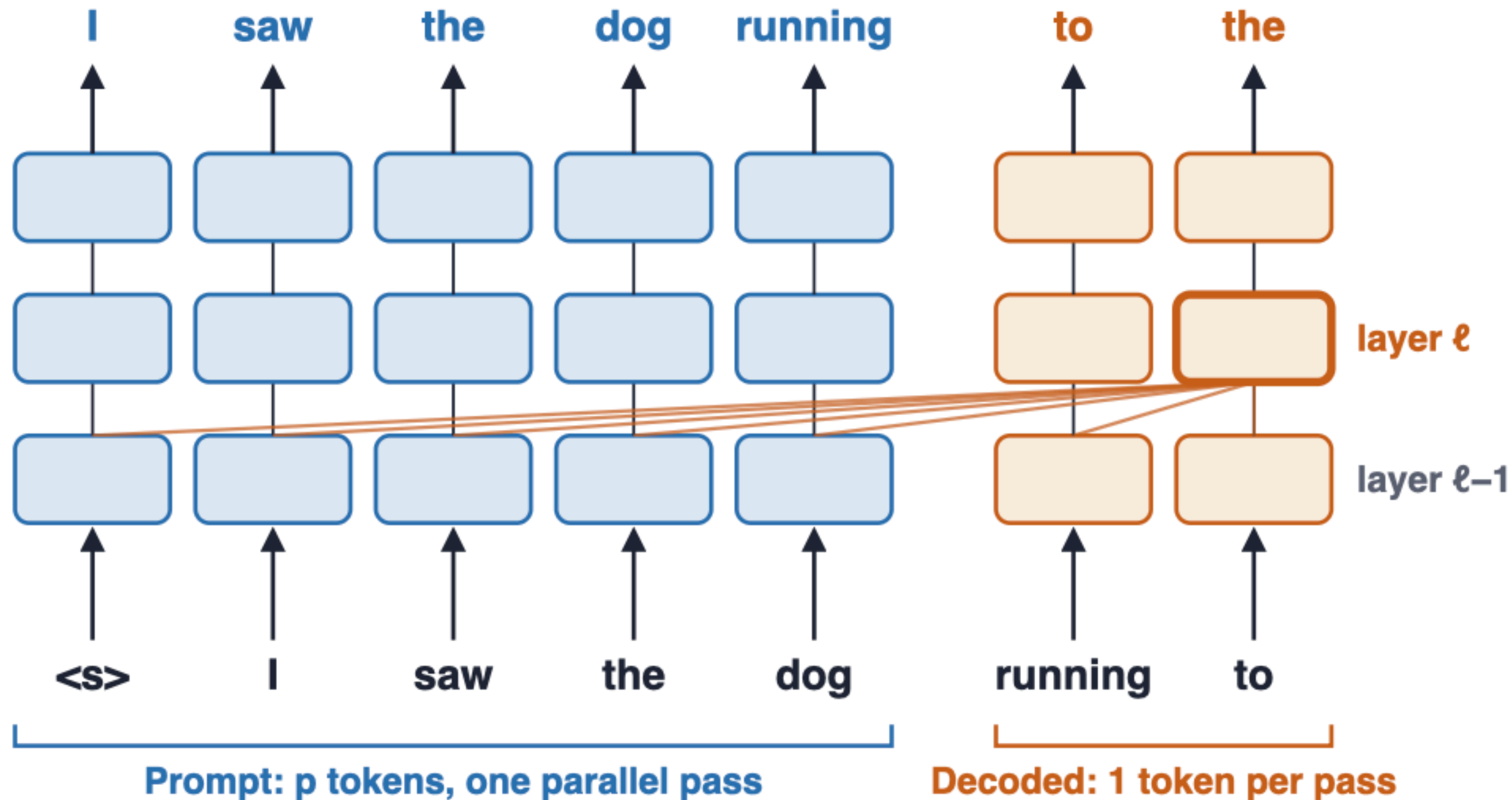
Also common

WordPiece (BERT) and Unigram (in SentencePiece, e.g. T5) build subword vocabularies with different rules.

What is decoding?

- ▶ Choosing a word to generate based on the model's probabilities.
- ▶ Autoregressive generation: repeatedly choosing the next word conditioned on previous generations.

Decoding Computation



Blue: computed once

The prompt runs through all L layers together. Each cell's keys and values are saved in the cache.

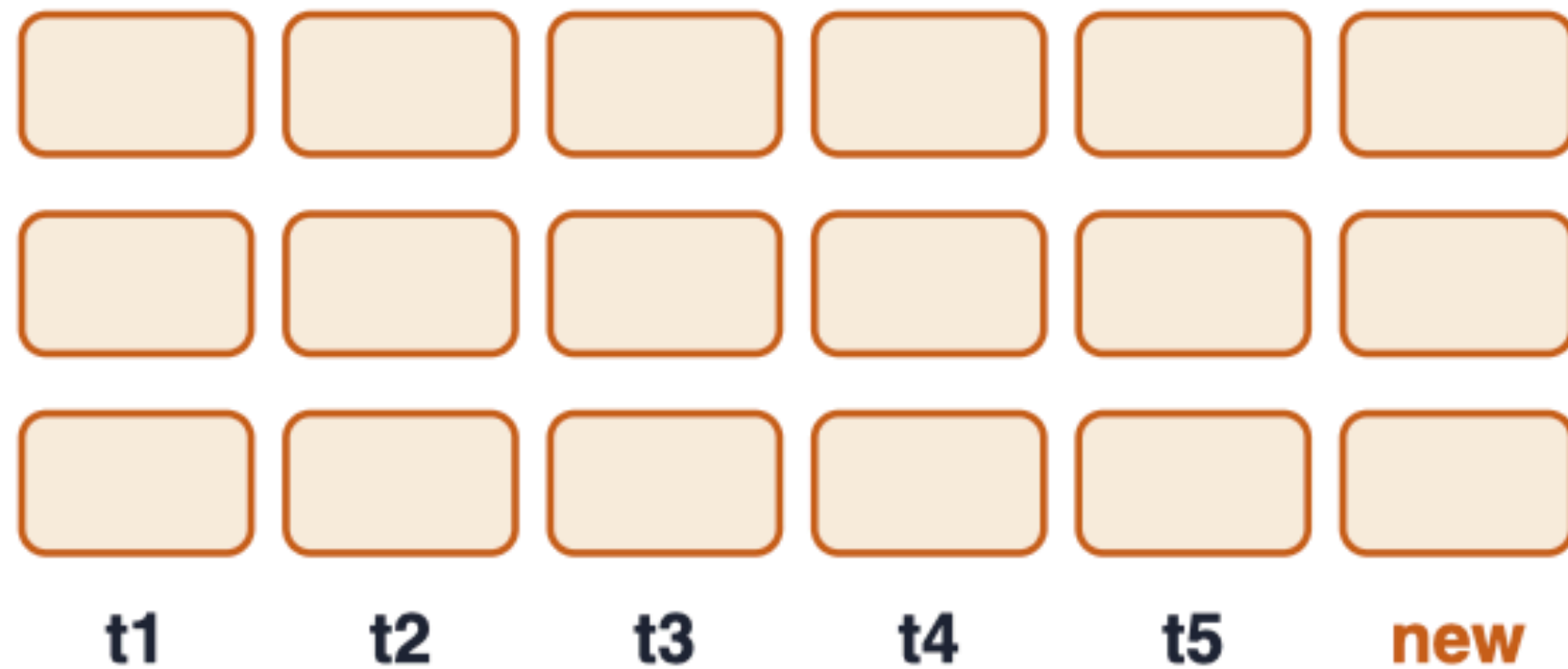
Orange: one column per step

Each predicted word becomes the next input. It enters at the bottom and makes one pass up through all L layers.

Decoding Computation

Without KV cache

Context of $p + i$ tokens at step i

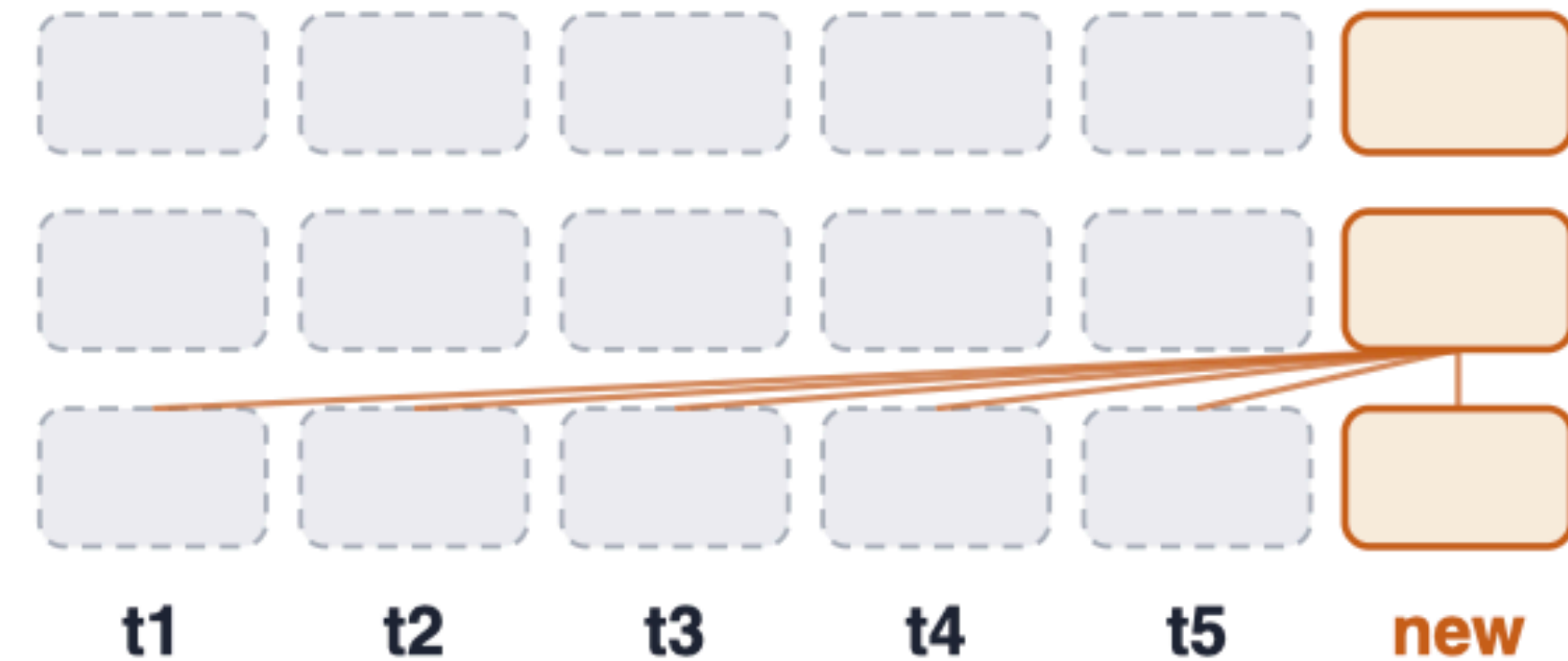


All $p+i$ tokens are rerun through L layers, and each attends to all the others.

Step i : $O((p+i)^2 \cdot L)$

With KV cache

Context of $p + i$ tokens at step i

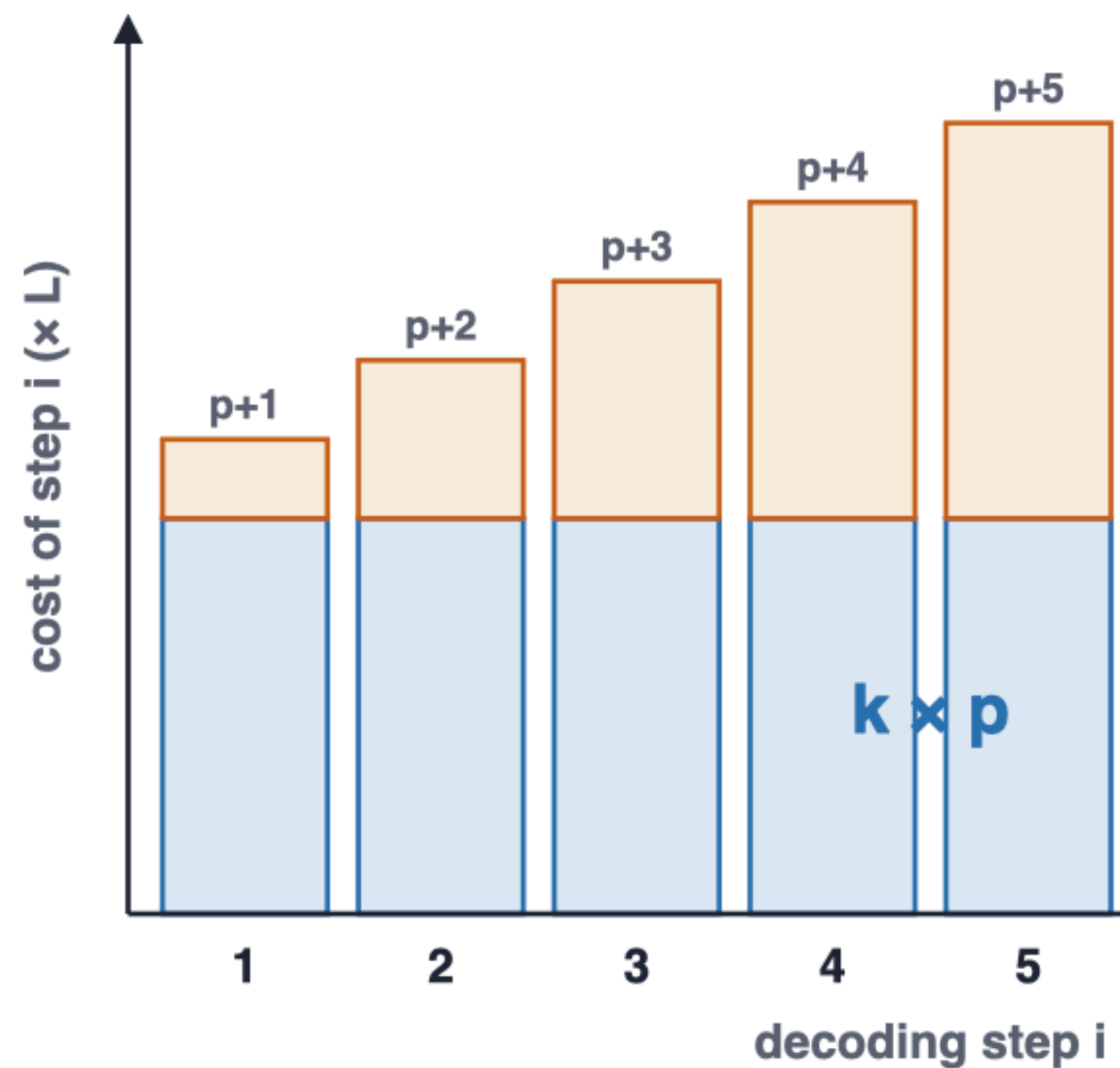


Only the new token runs through L layers. It reads the $p+i$ cached tokens.

Step i : $O((p+i) \cdot L)$

Decoding Computation

Adding up k decoding steps



Step i reads $p+i$ cached tokens in each of L layers, so its bar has height $p+i$.

Blue rectangle

Every step reads the whole prompt: $k \cdot p \cdot L$

Orange staircase

The growing tail: $(1+2+\dots+k) \cdot L \approx k^2 L / 2$

$$\text{Total} = O(k \cdot p \cdot L + k^2 \cdot L)$$

Decoding Computation

Total operations: with vs without KV cache

	Without KV cache	With KV cache
Prompt pass (prefill)	$O(p^2 \cdot L)$	$O(p^2 \cdot L)$
Decoding step i	$O((p+i)^2 \cdot L)$	$O((p+i) \cdot L)$
All k decoding steps	$\sum (p+i)^2 \cdot L = O(k \cdot p^2 \cdot L + k^2 \cdot p \cdot L + k^3 \cdot L)$	$\sum (p+i) \cdot L = O(k \cdot p \cdot L + k^2 \cdot L)$
Total	$\approx O(k \cdot (p+k)^2 \cdot L)$	$O(p^2 \cdot L + k \cdot p \cdot L + k^2 \cdot L) = O((p+k)^2 \cdot L)$

Example: $p = 1,000$ prompt tokens, $k = 1,000$ generated

$\approx 2.3 \text{ billion} \times L$

decoding work without the cache

Same example

$\approx 1.5 \text{ million} \times L$

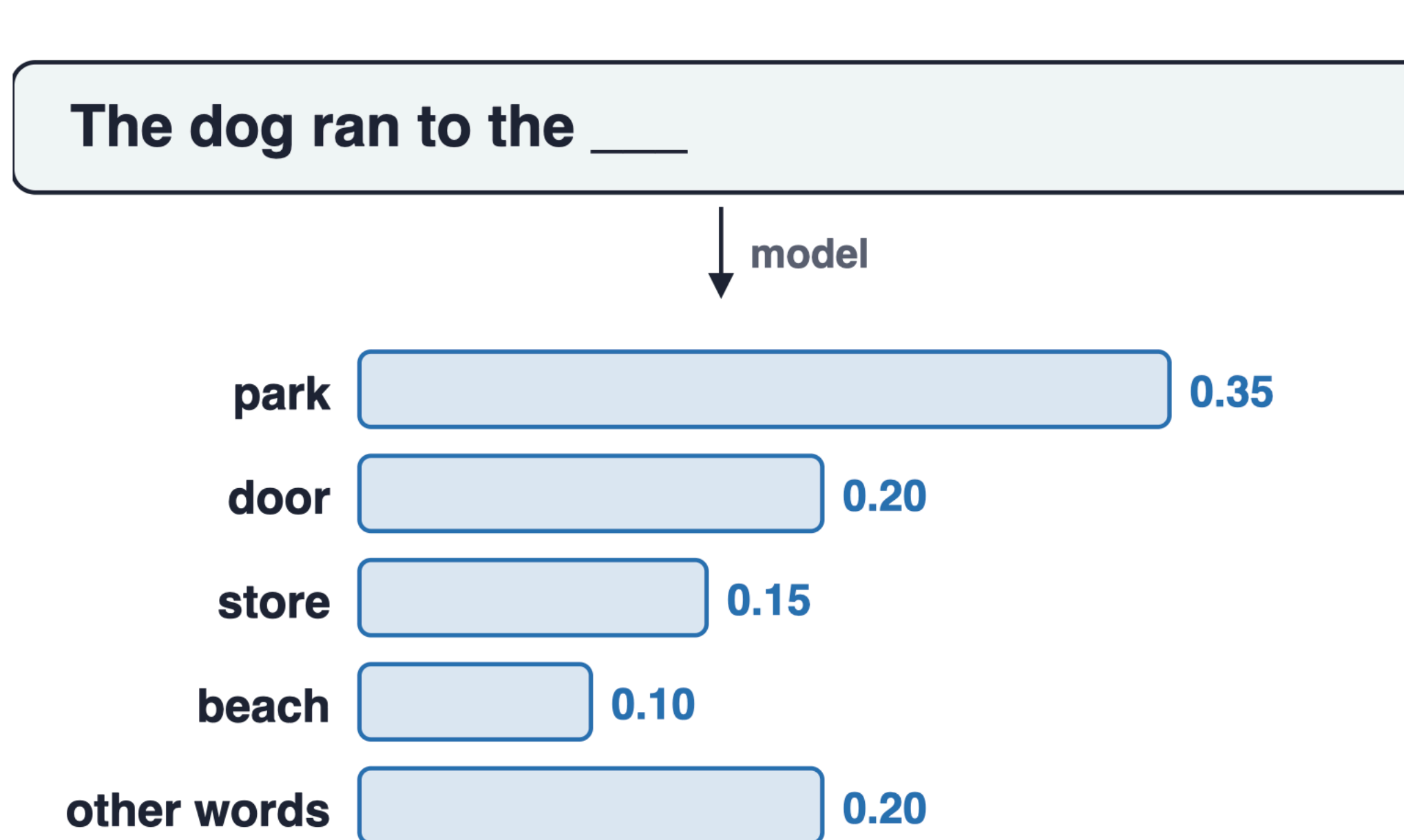
decoding work with the cache: about 1,500× less

Decoding Basics

- ▶ LMs place a distribution $P_{\theta}[y_i | y_1, \dots, y_{i-1}]$
- ▶ The model estimates distribution — how should we generate a sequence?

Decoding Basics

What do we want from decoding?



The model gives probabilities, not text.
How do we turn them into words?

A. The most probable text

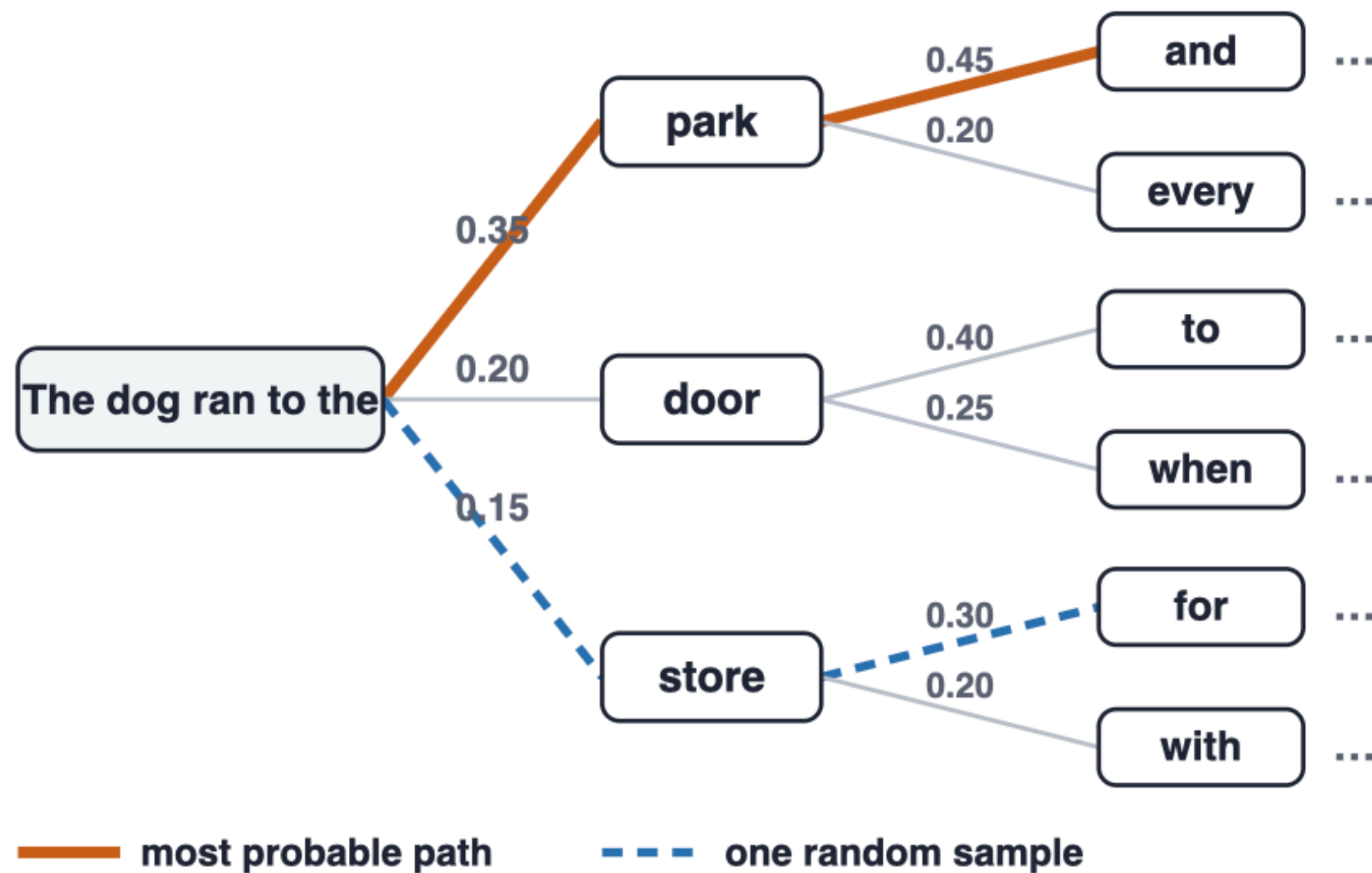
Search for the sequence the model scores highest.

B. Text that looks real

Produce something like what a person would plausibly write.

Decoding Basics

Two goals: search or sample



Most probable fits

Tasks with one right answer: translation, speech recognition, short factual answers.

Typical, varied fits

Open-ended tasks: stories, chat, brainstorming.

Is the most probable text always the best text?

Decoding Basics

- ▶ LMs place a distribution $P_{\theta}[y_i | y_1, \dots, y_{i-1}]$
- ▶ The model estimates distribution — how should we generate a sequence?
 - ▶ Option 1: Greedy decoding

Greedy decoding

- ▶ Choose the most-likely token at each step.

$$y_i = \operatorname{argmax}_y P_\theta[y|y_{<t}]$$

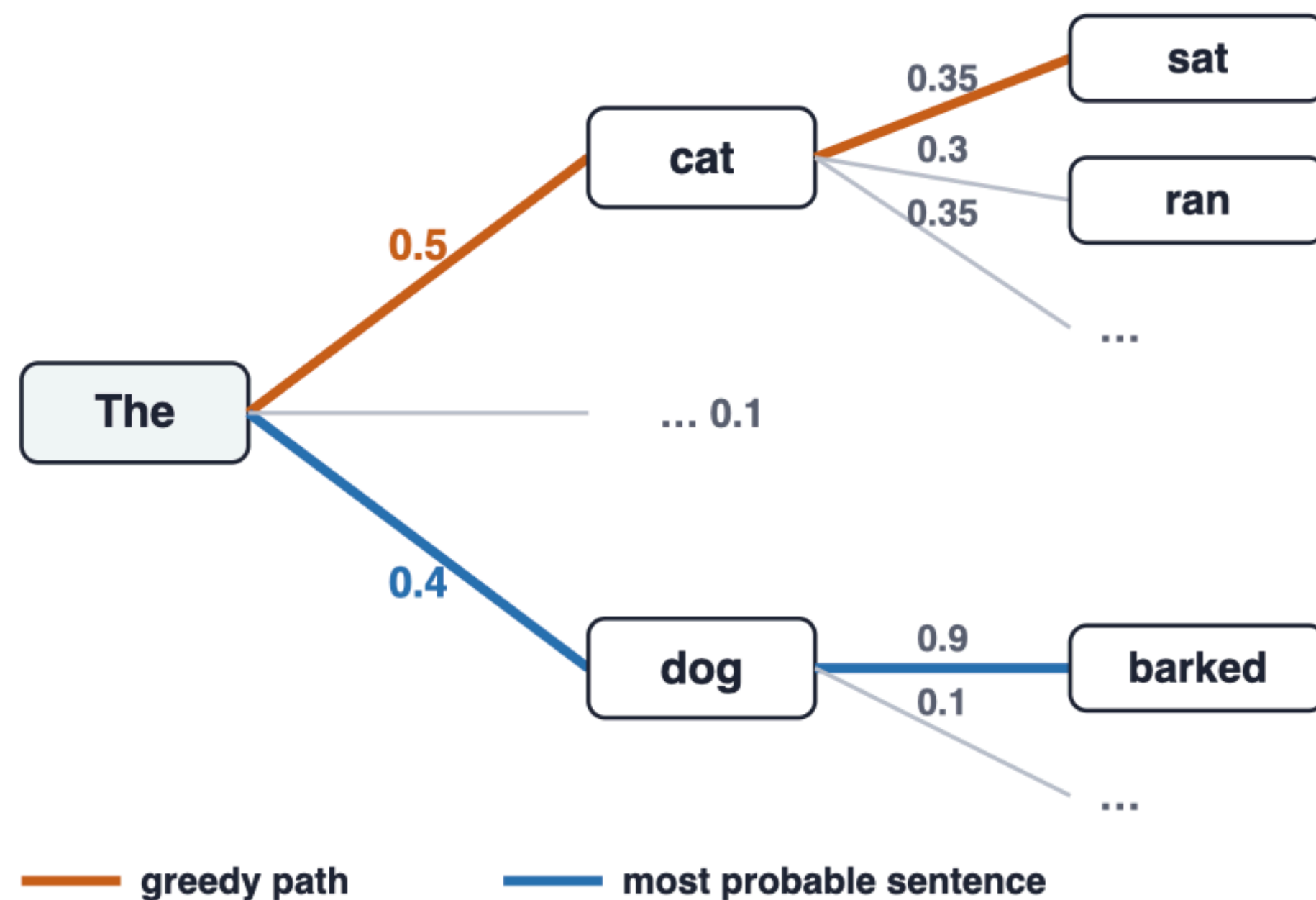
- ▶ When does it stop?

Greedy decoding

- Choose the most-likely token at each step.

$$y_i = \operatorname{argmax}_y P_\theta[y|y_{<t}]$$

Greedy can miss the most probable sentence



Greedy: The cat sat

$$0.5 \times 0.35 = \mathbf{0.175}$$

Best: The dog barked

$$0.4 \times 0.9 = \mathbf{0.36}$$

Why greedy misses it

It commits to "cat" before seeing that "dog" leads somewhere much likelier.

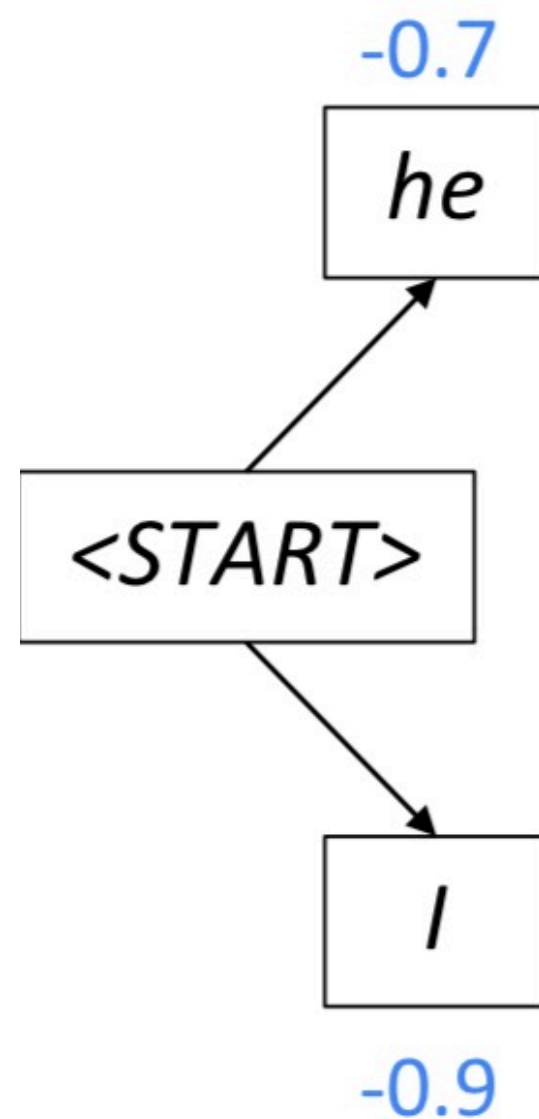
Beam-Search

- ▶ Width-limited breadth-first search
- ▶ Hyper-parameter: Size of beam (K)

Beam Search

- ▶ At each step, keep track of top K (beam size) hypotheses

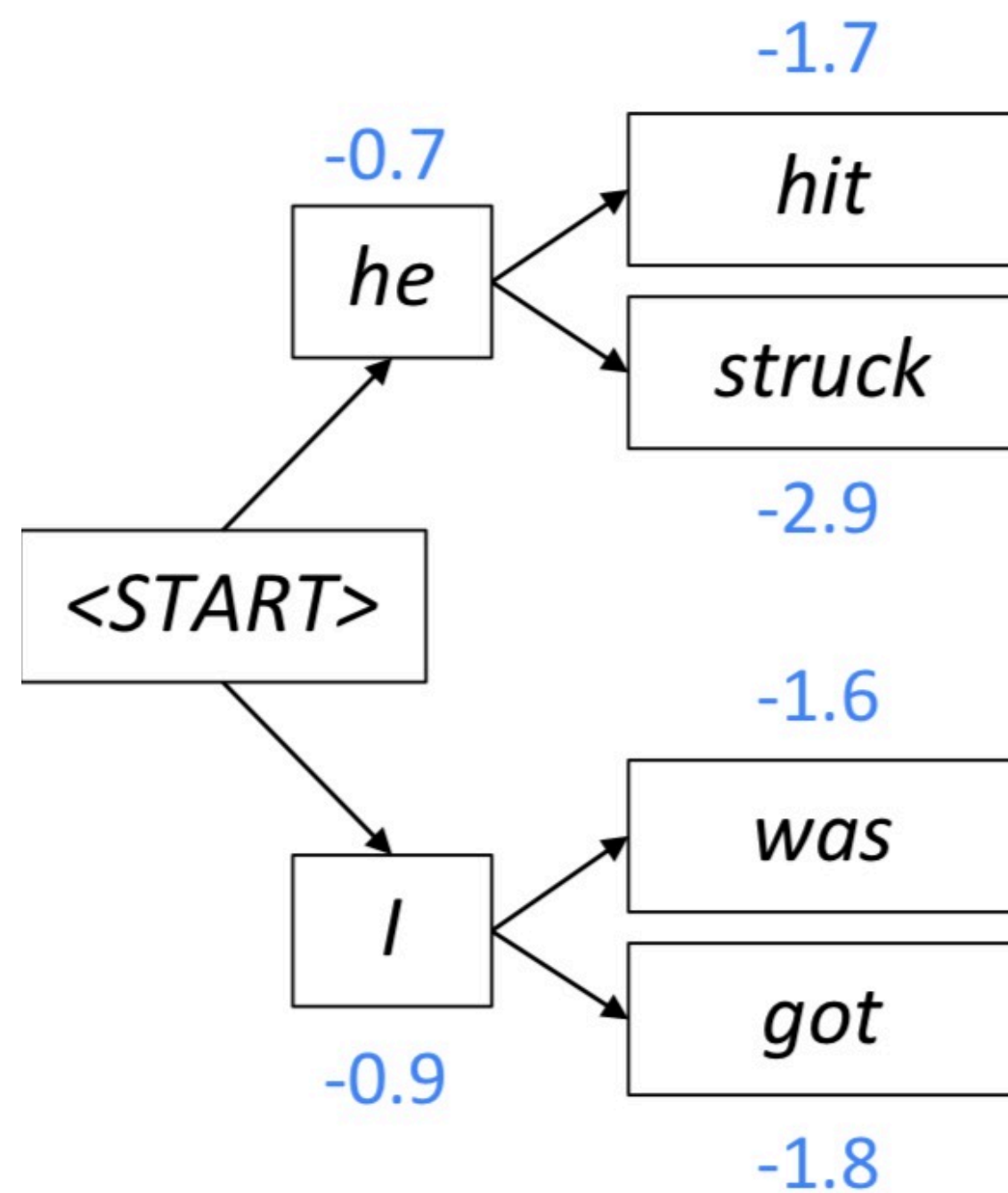
- ▶ Based on $\text{score}(y_1, y_2 \dots y_t) = \sum_{t=1}^t \log P(y_i | y_1, \dots y_{i-1})$



Beam Search

- ▶ At each step, keep track of top K (beam size) hypotheses

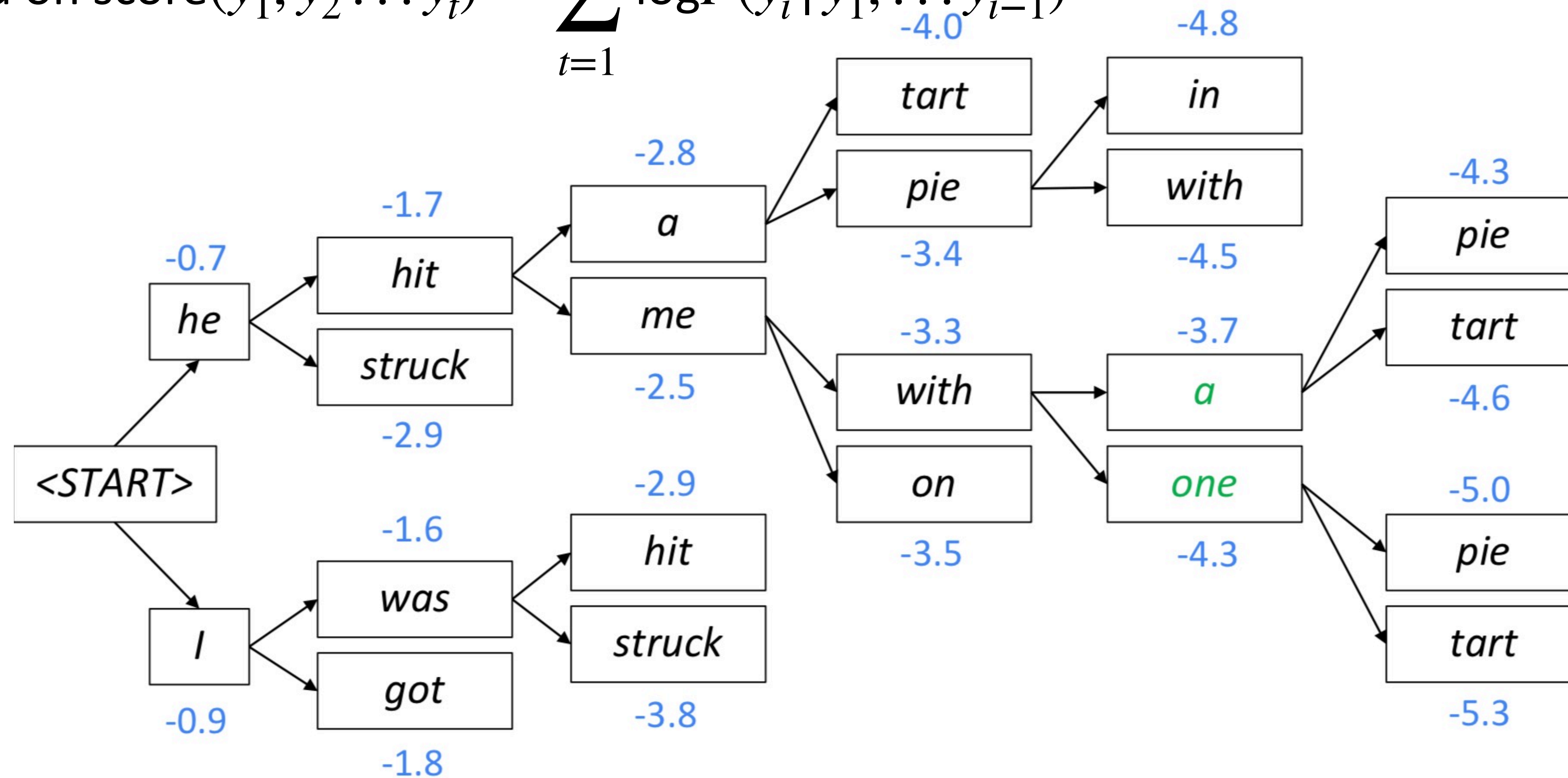
- ▶ Based on $\text{score}(y_1, y_2 \dots y_t) = \sum_{t=1}^t \log P(y_i | y_1, \dots y_{i-1})$



Beam Search

- At each step, keep track of top K (beam size) hypotheses

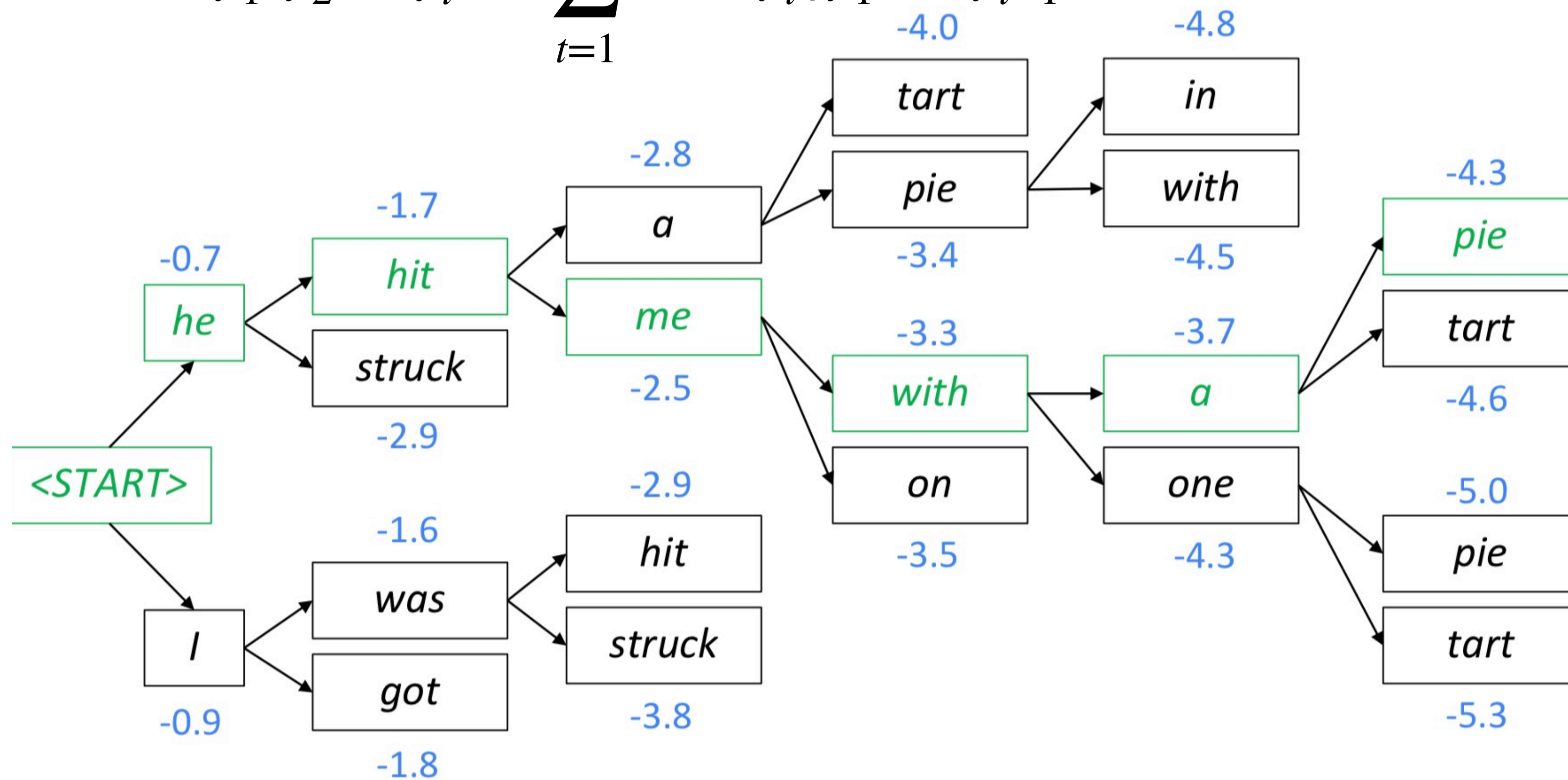
Based on $\text{score}(y_1, y_2 \dots y_t) = \sum_{i=1}^t \log P(y_i | y_1, \dots, y_{i-1})$



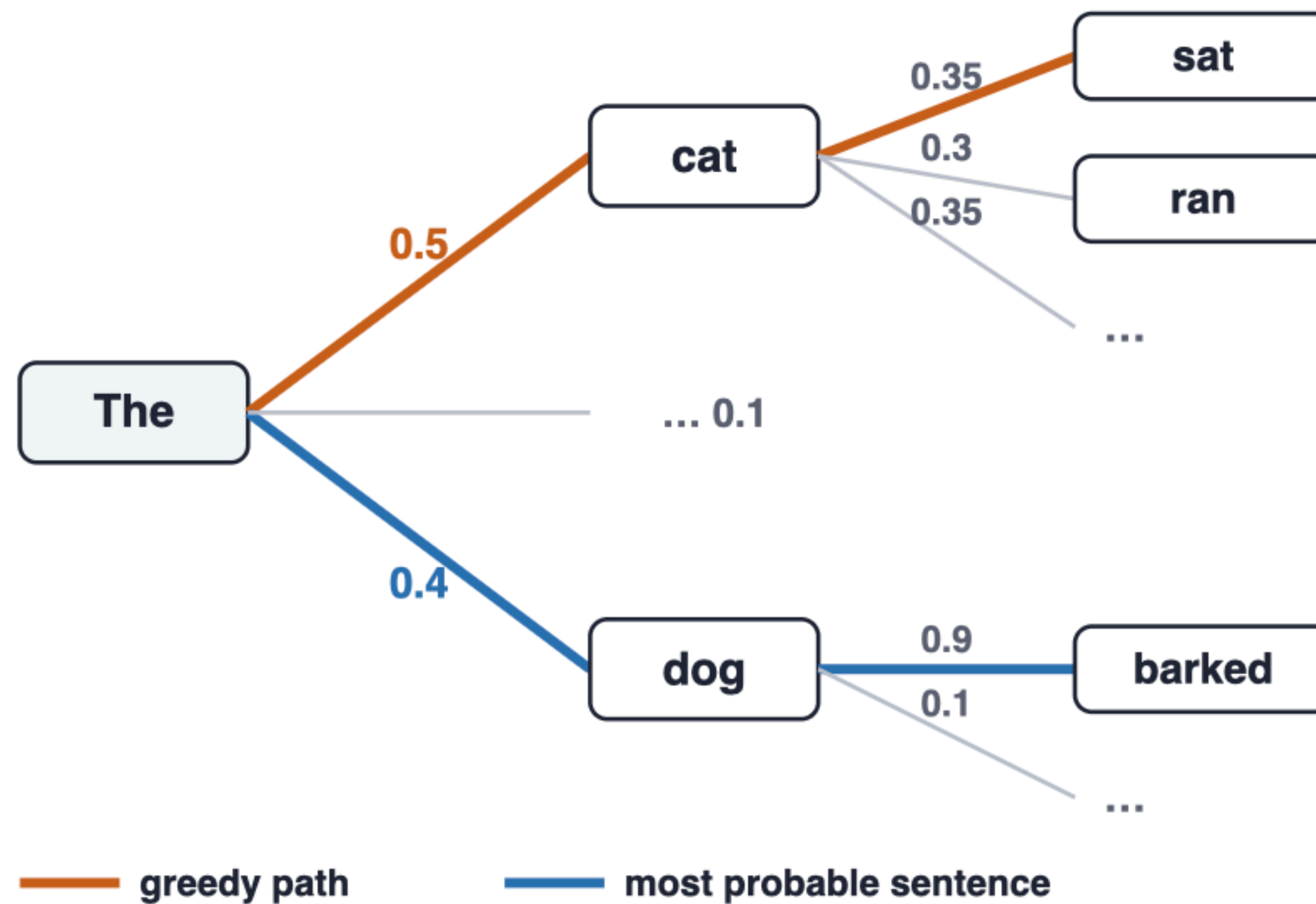
Beam Search

- At each step, keep track of top K (beam size) hypotheses

- Based on $\text{score}(y_1, y_2 \dots y_t) = \sum_{i=1}^t \log P(y_i | y_1, \dots, y_{i-1})$



Beam Search



Greedy: The cat sat

$$0.5 \times 0.35 = \mathbf{0.175}$$

Best: The dog barked

$$0.4 \times 0.9 = \mathbf{0.36}$$

Why greedy misses it

It commits to “cat” before seeing that “dog” leads somewhere much likelier.

**Beam size 2 keeps both “cat” and “dog”,
so it finds “The dog barked.”**

Benefits of beam search

- Typically shows gains over greedy decoding for closed-ended tasks such as question answering, translation

Model	Dataset	Metric		
			Greedy	BS
Llama2-7B	HumanEval MBPP	Pass@1	12.80	15.24
			17.80	19.40
	GSM8K	Acc	13.87	17.21
	XSUM CNN/DM	R-L	27.21	21.88
			23.43	20.69
	De⇒En	B-4	28.80	30.14
	En⇒De		22.63	23.99
	Zh⇒En		19.44	20.11
	En⇒Zh		15.15	14.50
	CQA SQA	Acc	62.90	64.37
			60.76	62.25

A Thorough Examination of Decoding Methods in the Era of LLMs
<https://arxiv.org/abs/2402.06925>

Degeneration

- ▶ Beam search fails because the model is *locally normalized*
- ▶ Let's look at all the individual decisions that get made here

$P(\text{Nacional} \mid \dots \text{Universidad})$ is high

$P(\text{Autónoma} \mid \dots \text{Universidad Nacional})$ is high

$P(\text{de} \mid \dots \text{Universidad Nacional Autónoma})$ is high

$P(\text{México} \mid \text{Universidad Nacional Autónoma de})$ is high

$P(/ \mid \dots \text{México})$ and $P(\text{Universidad} \mid \dots \text{México} /)$ — these probabilities may be low. But those are just 2/6 words of the repeating fragment

- ▶ **Each word is likely given the previous words but the sequence is bad**

Beam Search, $b=32$:

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

Holtzman et al. (2019)

Issues with “optimal” decoding

- ▶ Assume a biased coin $\Pr(H) = 0.6$, $\Pr(T) = 0.4$
- ▶ Most likely sequence would be:
 - HHHHHHHH.....
- ▶ But this outcome is atypical.

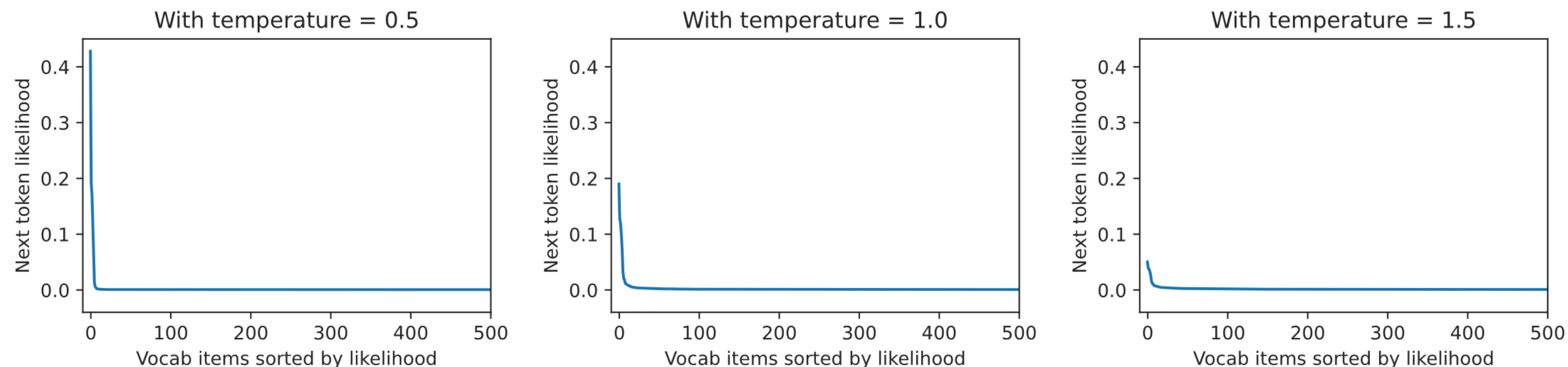
Locally typical sampling [Meister et al., 2022]

Sampling

- Given h is the vector with logits, and T is the temperature

$$p_{\theta}[x_t = w] = \frac{\exp(h_w/T)}{\sum_{w'} \exp(h_{w'}/T)}$$

- What happens with $T = 1$? $T = 0$ (or almost)? $T \in [0,1)$? $T > 1$?



Drawbacks of Sampling

- ▶ Sampling is “too random”

Pure Sampling:

They were cattle called Bolivian Cavalleros; they live in a remote desert uninterrupted by town and they speak huge, beautiful, paradisiacal Bolivian linguistic thing. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "They've only been talking to scientists, like we're being interviewed by TV

$P(y \mid \dots \text{they live in a remote desert uninterrupted by})$

0.01 roads

0.01 towns

0.01 people

0.005 civilization

...

0.0005 town

Good options, maybe accounting for 90% of the total probability mass. So a 90% chance of getting something good

Long tail with 10% of the mass

Holtzman et al. (2019)

Decoding Strategies

- ▶ Story generation (this is with GPT-2):

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Beam Search, $b=32$:

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

Pure Sampling:

They were cattle called Bolivian Cavalleros; they live in a remote desert uninterrupted by town, and they speak huge, beautiful, paradisiacal Bolivian linguistic thing. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavalleros."

- ▶ Beam search tends to generate generic, repetitive, awkward

- ▶ Sampling is too noisy — introduces many grammatical errors

Holtzman et al. (2019)

Truncation Sampling

$P(y \mid \dots \text{they live in a remote desert uninterrupted by})$

0.01 roads

0.01 towns

0.01 people

0.005 civilization

—————→ renormalize and sample

cut off after top K tokens

cut off after $p\%$ of mass

- ▶ Define a threshold
 - ▶ *Top K scored tokens or Top P probability mass*
- ▶ Typically top K = 5, Top P = 0.85,.....

Variants of Truncation Sampling

- ▶ Top-k: Sample from k-most probable
- ▶ Top-p: Cumulative probability at most p
- ▶ η : Min prob. Proportional to entropy
- ▶ Epsilon: minimum probability
- ▶ Min-p: Prob. At least $p_{\min}$, scaled to the max token prob

How to evaluate decoding algorithms?

- ▶ Performance:
 - ▶ Task accuracy
- ▶ Efficiency
 - ▶ Speed:
 - Latency: How long does it take to generate a response
 - Throughput: How many requests can be completed per second?
- ▶ Memory:
 - How much data do we have to keep on-device?

Exercise

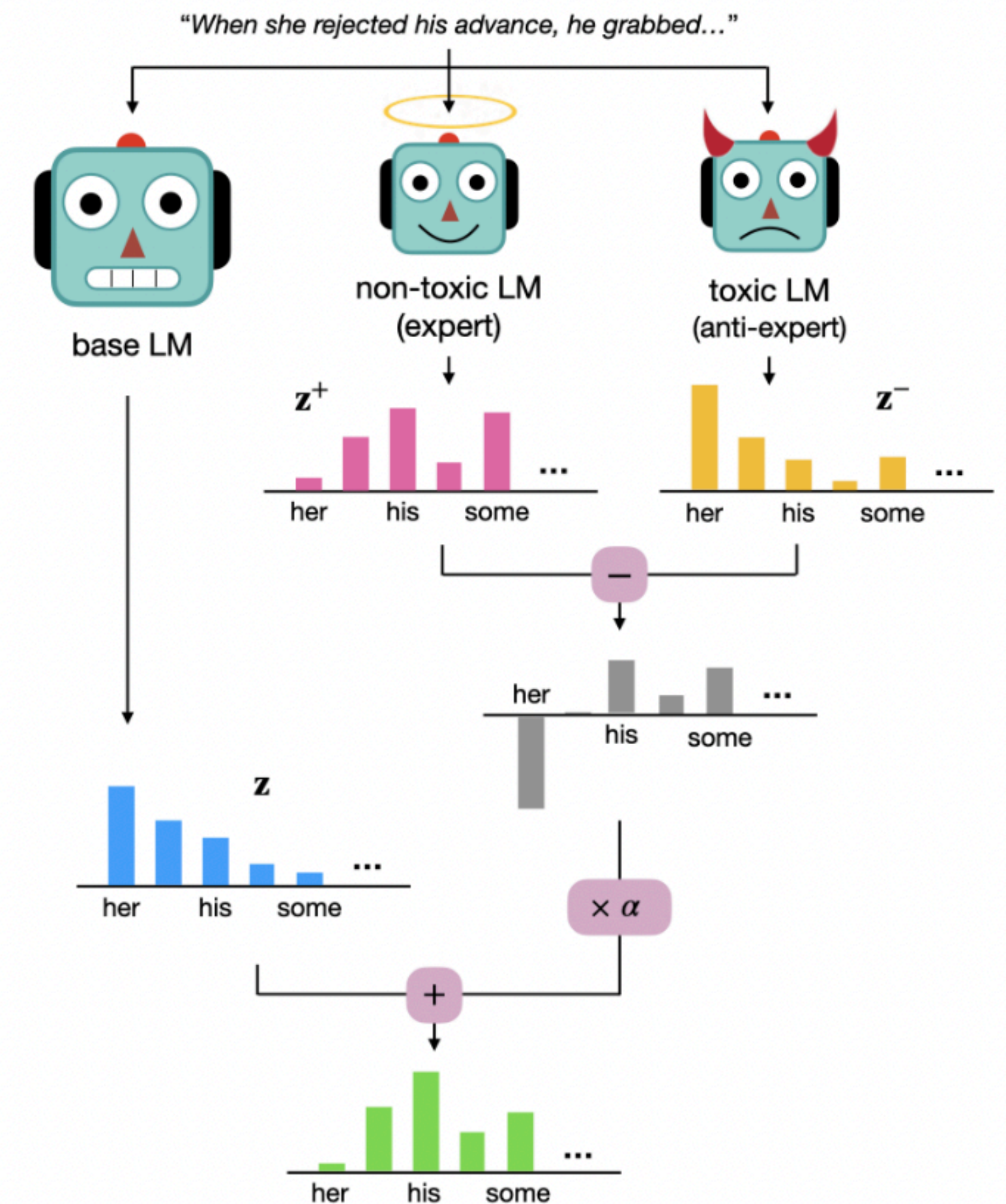
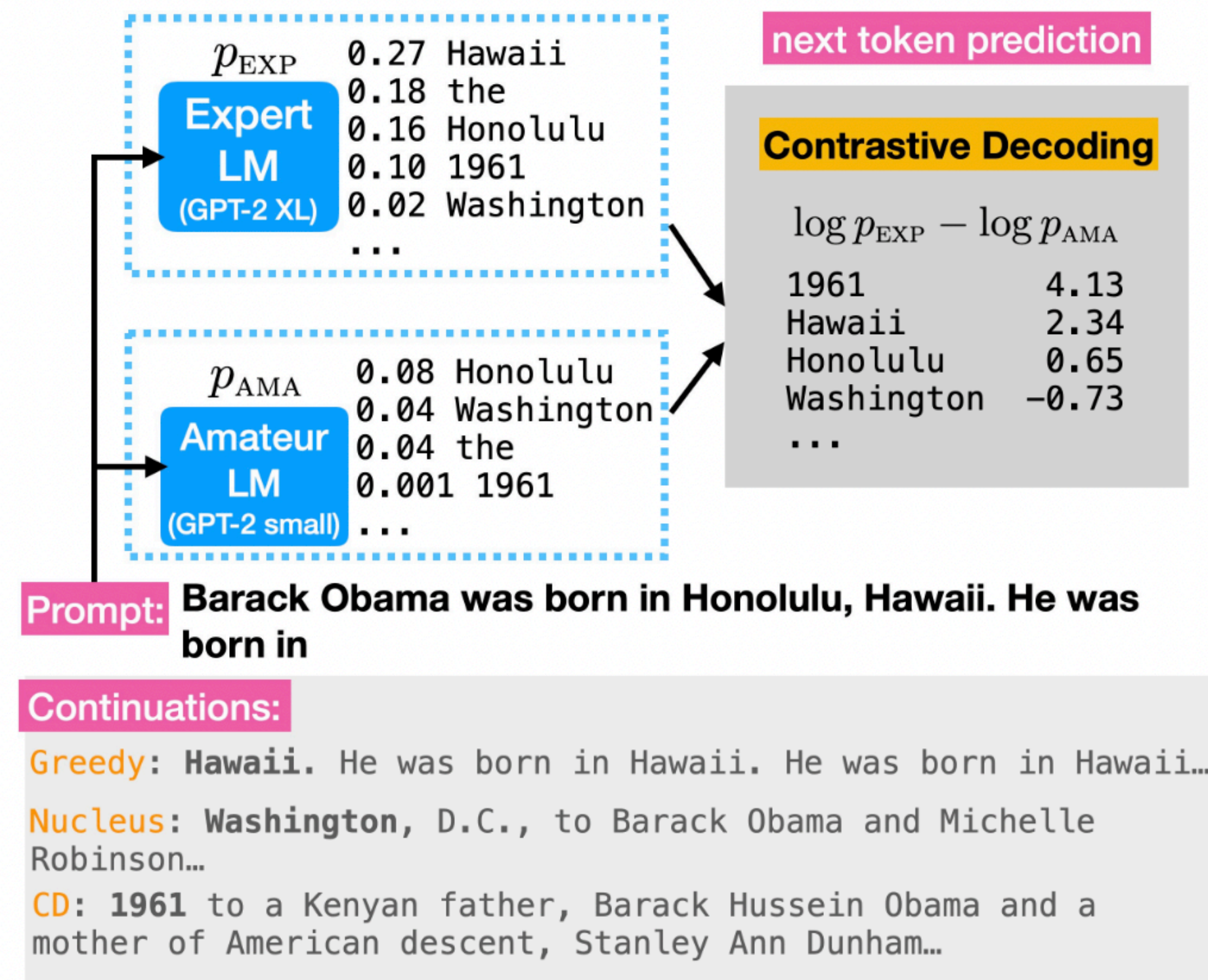
- ▶ You are generating a 5 word sentence from your language model, and your vocabulary size is 100.
- ▶ What is the size of the set of all possible sentences?
- ▶ How many forward passes is needed for the following decoding methods?
 - ▶ Greedy decoding: 5 forward passes
 - ▶ Beam search with beam size 1:
 - ▶ Beam search with beam size 10:
 - ▶ Top-p Sampling with $P=0.9$

Today

- ▶ Tips for Final Project Brainstorming
- ▶ Decoding with Language Model
 - ▶ Decoding Basics
 - ▶ **Advanced Decoding methods**
- ▶ Variants of Language Model
- ▶ Tokenization

Multi-LM Decoding

- Using multiple LMs (and their distribution) to derive a new distribution



DEXPERTS: Decoding-Time Controlled Text Generation with Experts and Anti-Experts <https://arxiv.org/abs/2105.03023>

Constrained Decoding

- ▶ Structural constraints (e.g., valid JSON format, valid email addresses)
 - ▶ Methods typically uses a parser, which filters next-token candidates by only allowing valid continuations
- ▶ Lexical constraints (must have specific words or must not have them)
 - ▶ Modified beam search, taking into consideration constraints

NeuroLogic Decoding: (Un)supervised Neural Text Generation with Predicate Logic Constraints [Lu et al, NAACL 21]

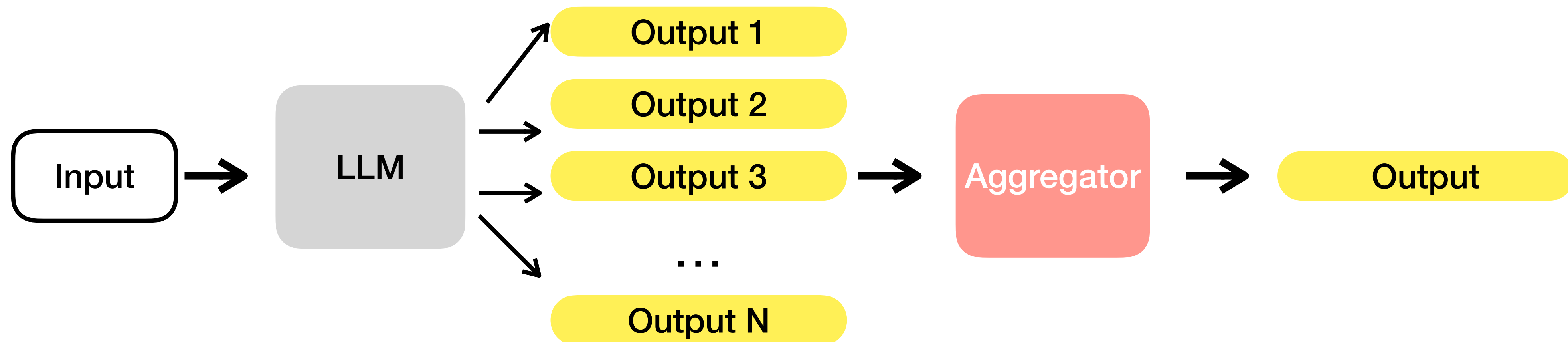
Parallel meta-generator

- Sample multiple trajectories in parallel: $y^1, y^2, \dots, y^N$

Each y^i is a decoded sequence of tokens

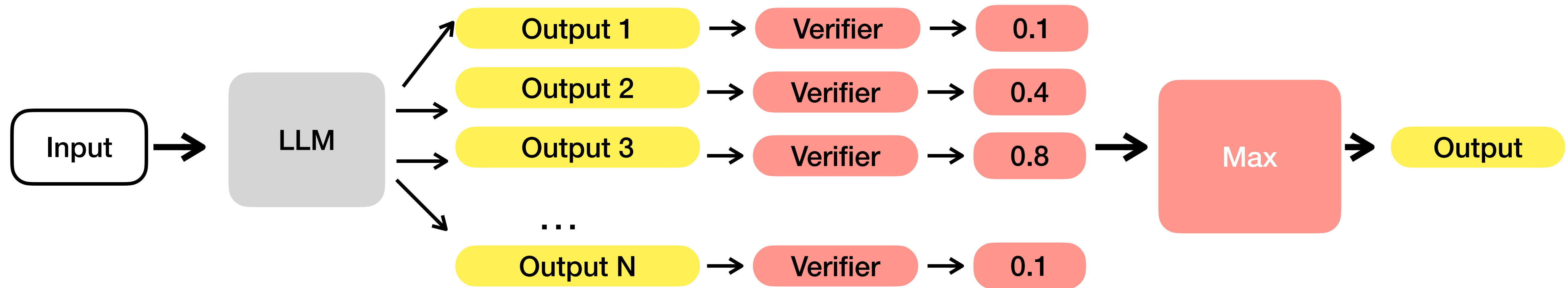
- Merge the resulting states to reach the final generation sequence:
 $y = h(y^1, y^2, \dots, y^N)$

h is your meta-generator



Aggregation Strategies with Verifier: Max

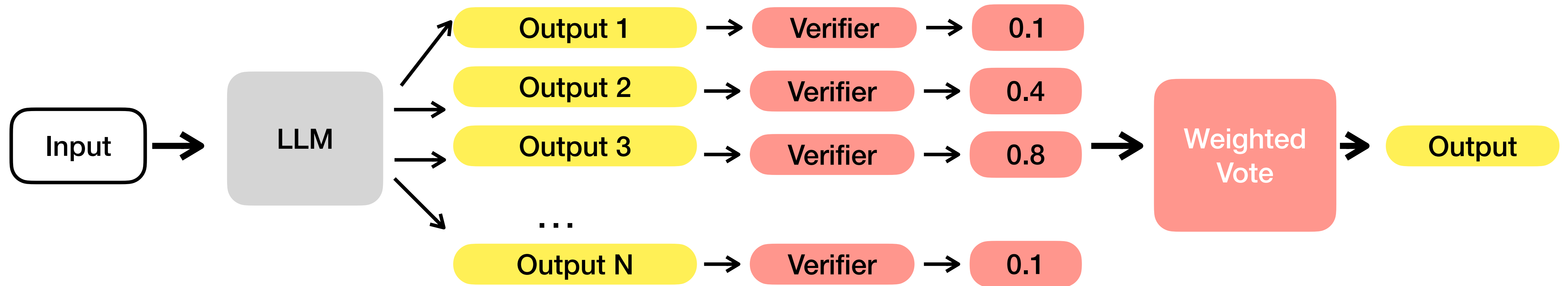
- Assume a verifier v that can score each output.



$$\operatorname{argmax}_{\{y^1, y^2, \dots, y^N\}} v(y)$$

- If you have a good verifier, performance improves as the number of generations N increases

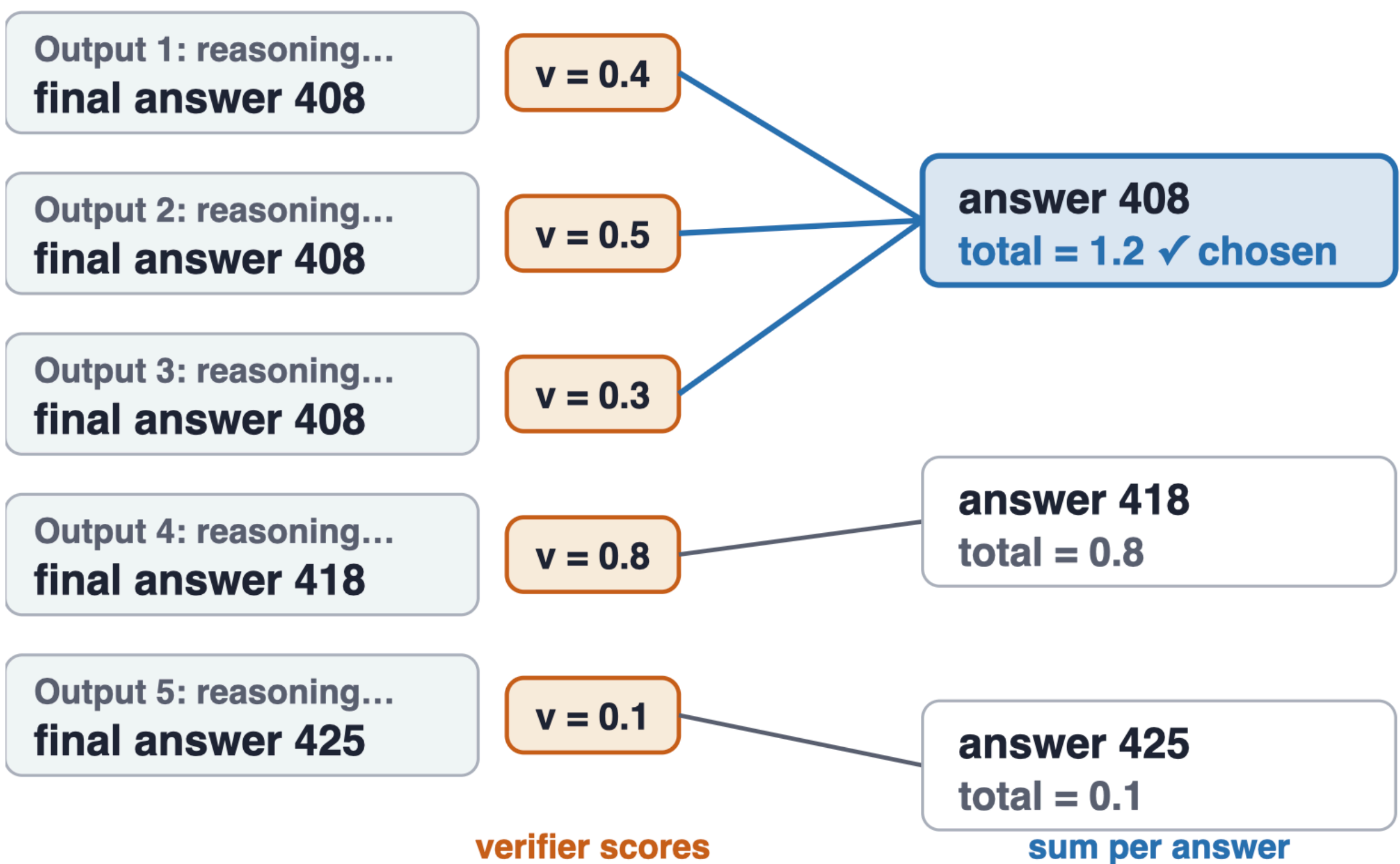
Aggregation Strategies with Verifier: Weighted Vote



$$\operatorname{argmax}_a \sum_{i=1}^N v(y^i) \cdot 1\{y^i = a\}$$

Aggregation Strategies with Verifier: Weighted Vote

Weighted vote: add up verifier scores per answer



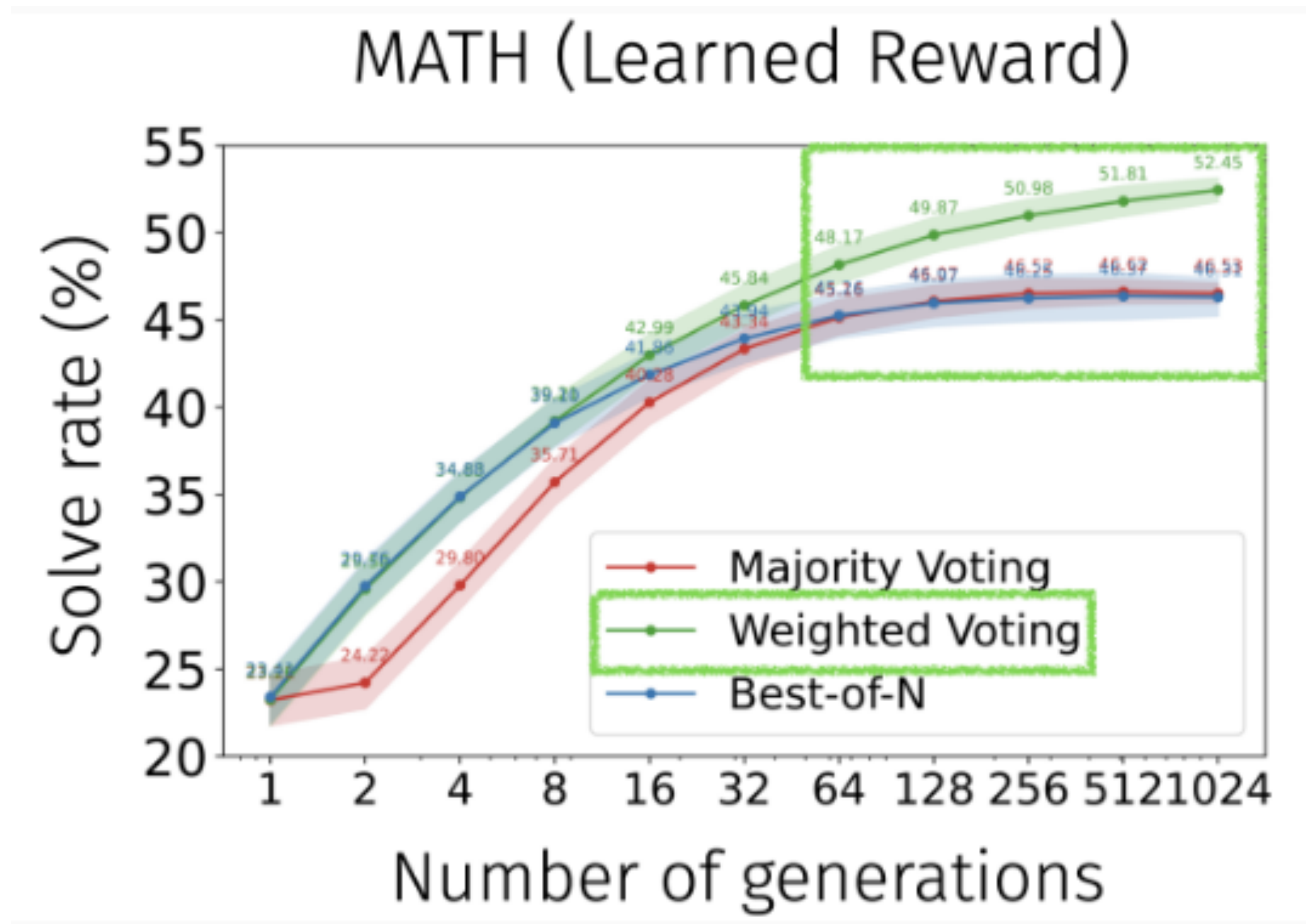
$$\text{answer} = \operatorname{argmax}_a \sum_i v(y_i) \cdot 1\{y_i \rightarrow a\}$$

$1\{y_i \rightarrow a\}$ is 1 if output i 's final answer is a .
Different reasoning, same answer: same bucket.

Strategy	Picks	Here
Majority vote	most common answer	408
Max (best-of-N)	single top-scored output	418 ×
Weighted vote	answer with top total score	408

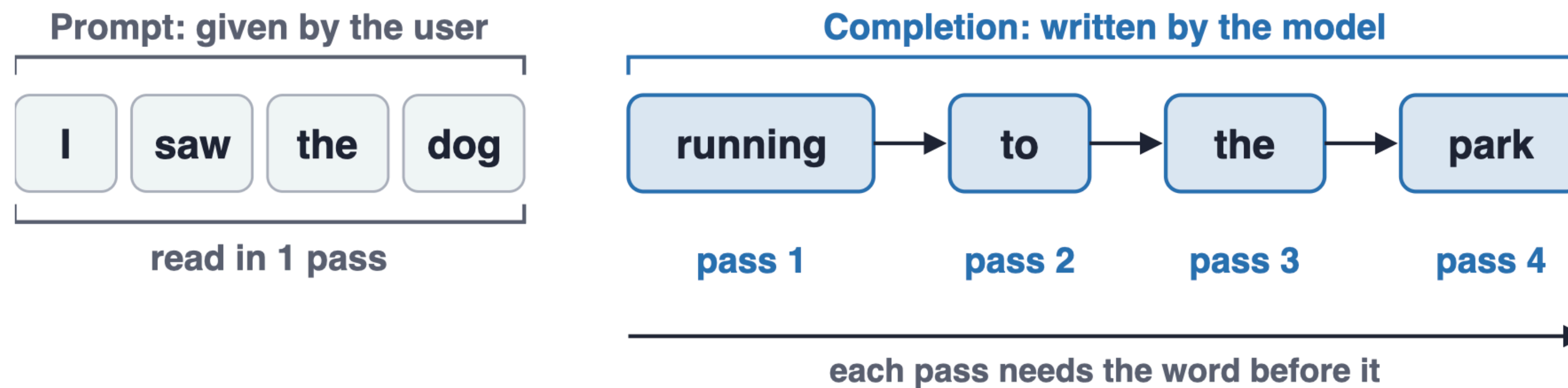
Example: “What is 17×24 ?” (correct answer 408). Scores are illustrative.

Aggregation Strategies



Speculative Decoding

Why generation is slow



Prompt

Given. The model reads all of it in one pass.

Completion

Generated one word at a time. With a big model, every word is one slow pass, and passes can't overlap.

Goal: the same completion from the big model, with fewer slow passes.

Speculative Decoding

Idea: guess with a small model, verify with the big one

Prompt

I saw the dog

Completion so far

running

Like an assistant and an editor

The assistant drafts quickly; the editor reads the draft once and fixes the first mistake.

Small model (fast) guesses the next 4 words:

to the house quickly

Big model reads the guess ONCE and fixes the first mistake:

to the ~~house~~ ~~quickly~~
✓ keep ✓ keep ✗ → "park" dropped

Completion now: running to the park (+3 words, 1 big-model pa

Speculative Decoding

Idea: guess with a small model, verify with the big one

Prompt

I saw the dog

Completion so far

running

Like an assistant and an editor

The assistant drafts quickly; the editor reads the draft once and fixes the first mistake.

Small model (fast) guesses the next 4 words:

to the house quickly

Big model reads the guess ONCE and fixes the first mistake:

to the ~~house~~ ~~quickly~~
✓ keep ✓ keep ✗ → "park" dropped

Completion now: running to the park (+3 words, 1 big-model pa

Speculative Decoding

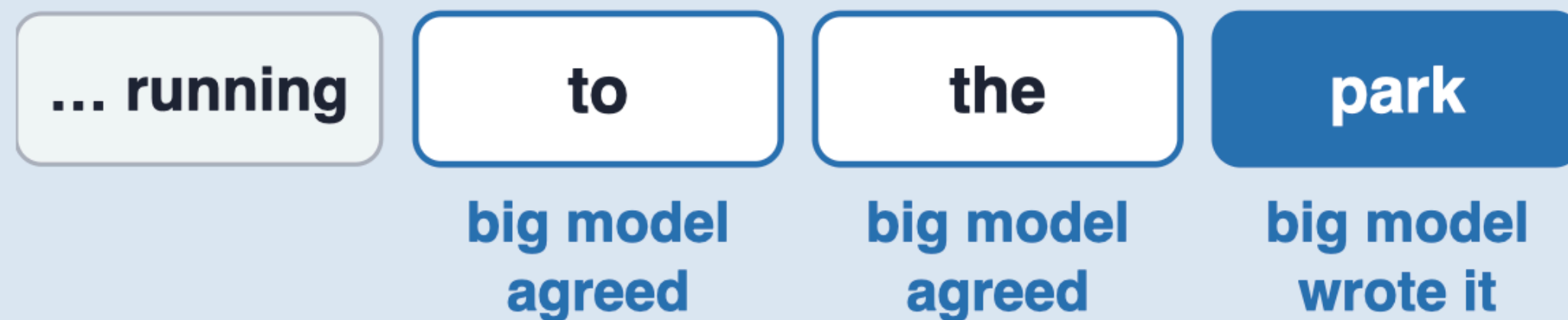


- ① The small model drafts 4 words.
- ② The big model reads them in one pass and gives its own word for each slot.
- ③ Keep the matches; at the first mismatch take the big model's word. Repeat from "park".

Speculative Decoding

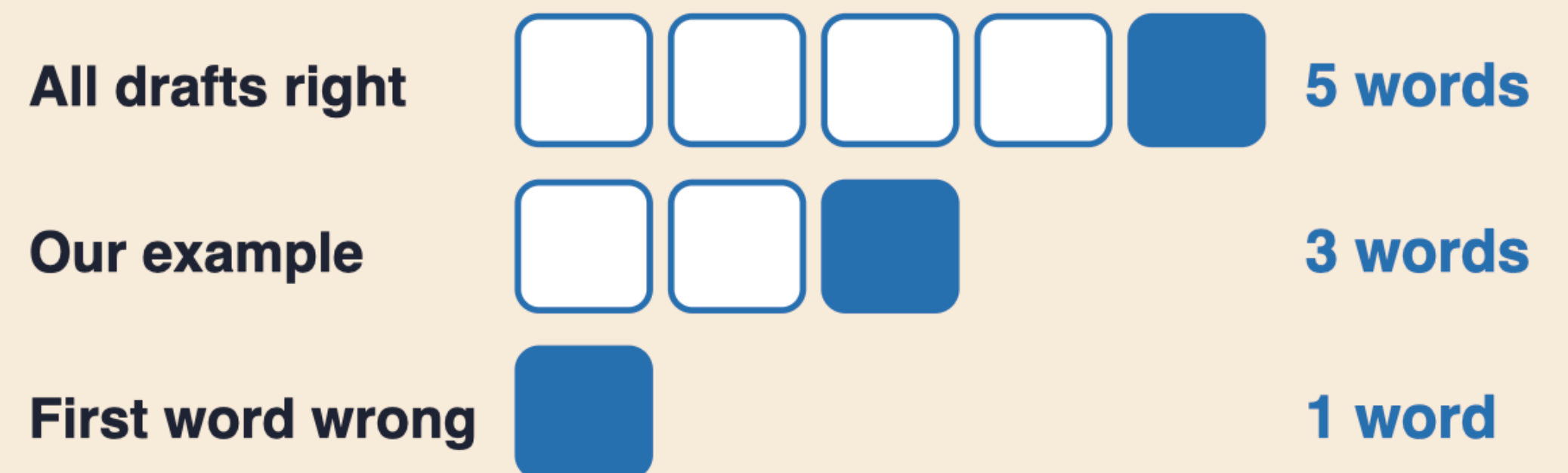
Why it works

Correct: same text



Every kept word is one the big model agreed with or wrote itself. The small model never decides anything.

Faster: several words per slow step



Filled tile: the big model's own word. Worst case is 1 word per step, like normal. Leviathan et al. report about 2–3× speed-ups.

Summary

- ▶ Decoding algorithm impacts the quality of generated text significantly, even with the same model parameters